

Chapter 5: Thread-Level Parallelism – Part 1

Introduction

What is a parallel or multiprocessor system?

Why parallel architecture?

Performance potential

Flynn classification

Communication models

Architectures

Centralized shared-memory

Distributed shared-memory

Parallel programming

Synchronization

Memory consistency models

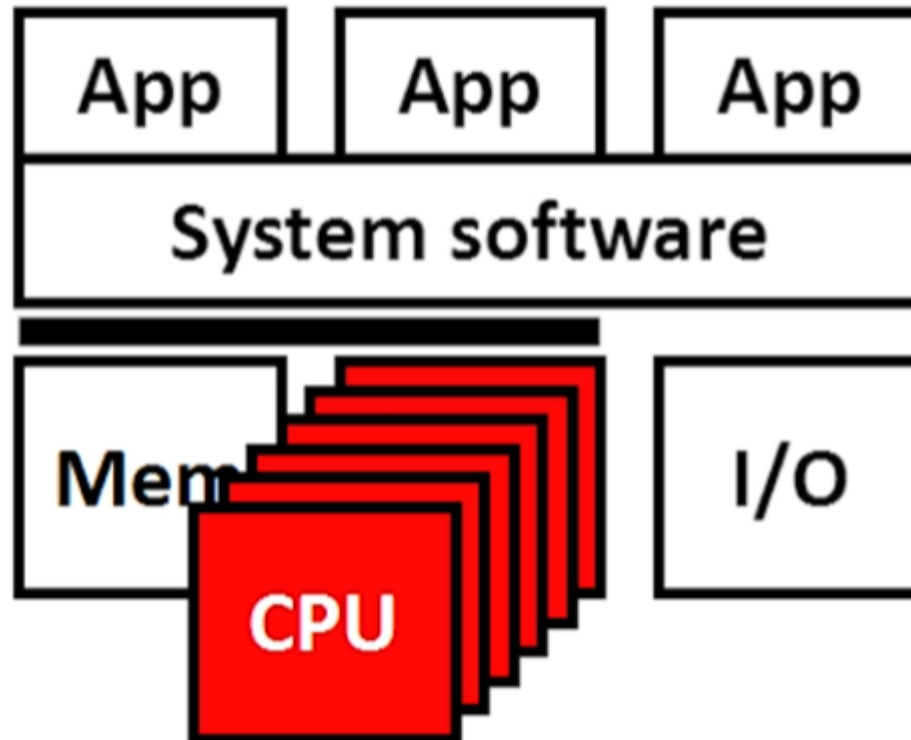
What level Parallelism?

- **Bit level parallelism: 1970 to ~1985**
 - 4 bits, 8 bit, 16 bit, 32 bit microprocessors
- **Instruction level parallelism (ILP): ~1985 through today**
 - Pipelining
 - Superscalar
 - VLIW
 - Out-of-Order execution
 - Limits to benefits of ILP?
- **Process Level or Thread level parallelism; mainstream for general purpose computing**
 - Servers are parallel
 - Highend Desktop dual processor PC

What is a parallel or multiprocessor system?

Multiple processor units working together to solve the same problem

Key architectural issue: Communication model



Why parallel architectures?

Absolute performance

Technology and architecture trends

Dennard scaling, ILP wall, Moore's law

⇒ Multicore chips

Connect multicore together for even more parallelism

Why Multiprocessors?

Microprocessors as the fastest CPUs

- Collecting several much easier than redesigning 1

Complexity of current microprocessors

- Do we have enough ideas to sustain 1.5X/yr?
- Can we deliver such complexity on schedule?

Slow (but steady) improvement in parallel software (scientific apps, databases, OS)

Emergence of embedded and server markets driving microprocessors in addition to desktops

- Embedded functional parallelism, producer/consumer model
- Server figure of merit is tasks per hour vs. latency

Performance Potential

Amdahl's Law is pessimistic

Let s be the serial part

Let p be the part that can be parallelized n ways

Serial: SSPPPPPP

6 processors: SSP

P

P

P

P

P

Speedup = $8/3 = 2.67$

$$T(n) = \frac{1}{s+p/n}$$

$$\text{As } n \rightarrow \infty, T(n) \rightarrow \frac{1}{s}$$

Pessimistic

Performance Potential (Cont.)

Gustafson's Corollary

Amdahl's law holds if run same problem size on larger machines

But in practice, we run larger problems and "wait" the same time

Performance Potential (Cont.)

Gustafson's Corollary (Cont.)

Assume for larger problem sizes

Serial time fixed (at s)

Parallel time proportional to problem size (truth more complicated)

Old Serial: SSPPPPPP

6 processors: SSPPPPPP

PPPPPP

PPPPPP

PPPPPP

PPPPPP

PPPPPP

Hypothetical Serial:

SSPPPPPP PPPPPP PPPPPP PPPPPP PPPPPP PPPPPP

$$\text{Speedup} = (8 + 5 \cdot 6) / 8 = 4.75$$

$$T'(n) = s + n \cdot p; T'(\infty) \rightarrow \infty!!!!$$

How does your algorithm "scale up"?

Flynn classification

Single-Instruction Single-Data (SISD)

- Uniprocessors

Multiple-Instruction Single-Data (MISD)

- ???; multiple processors on a single data stream

Single-Instruction Multiple-Data (SIMD)

- Examples: Illiac-IV, CM-2
 - Simple programming model; - Low overhead; - Flexibility; - All custom integrated circuits
- (Phrase reused by Intel marketing for media instructions ~ vector)

Multiple-Instruction Multiple-Data (MIMD)

- Examples: Sun Enterprise 5000, Cray T3D, SGI Origin
 - Flexible; -Use off-the-shelf micros

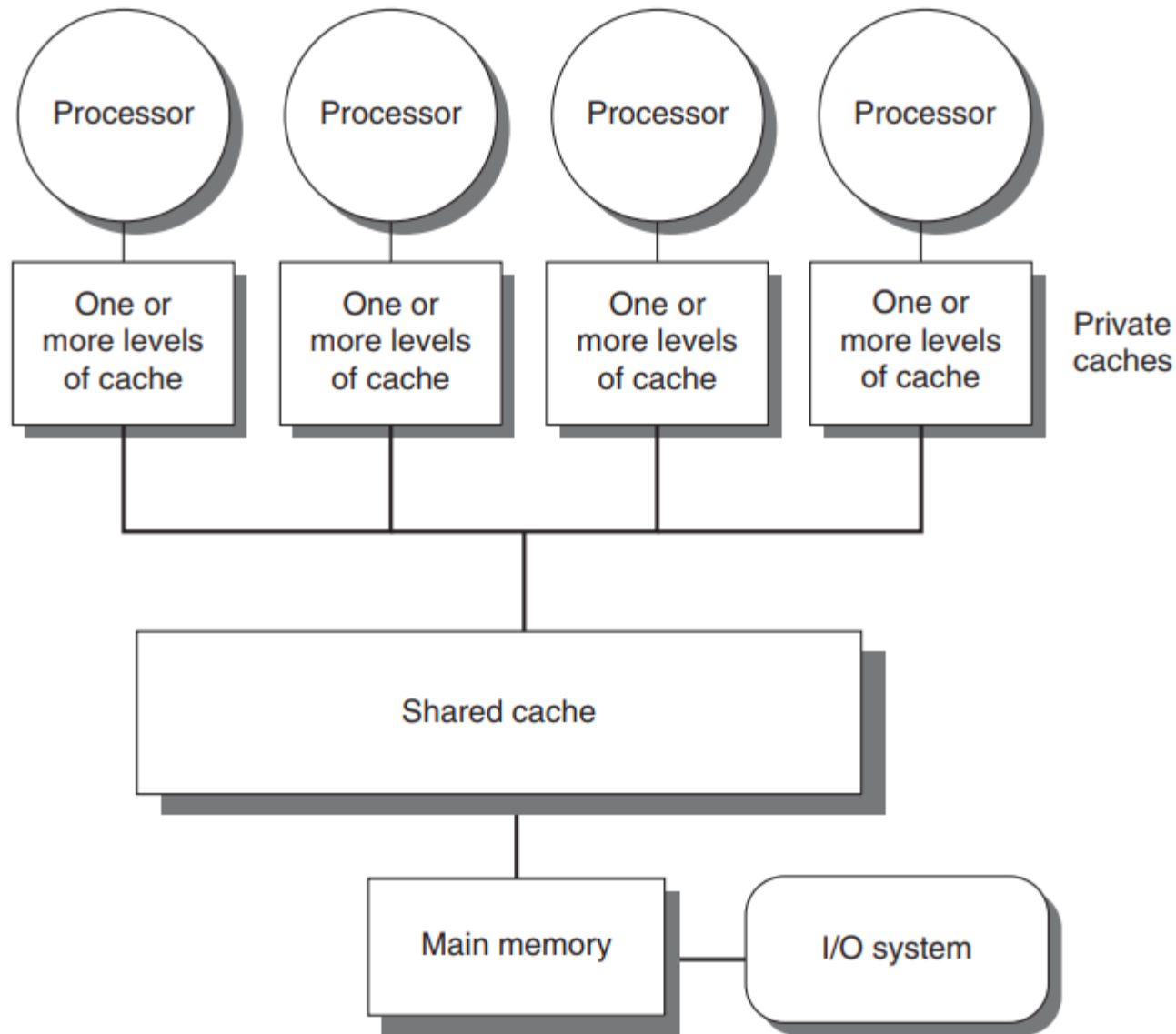
Major MIMD Styles

1. **Centralized shared memory ("Uniform Memory Access" time or "Shared Memory Processor")**
2. **Decentralized memory (memory module with CPU)**
 - get more memory bandwidth, lower memory latency
 - Drawback: Longer communication latency
 - Drawback: Software model more complex

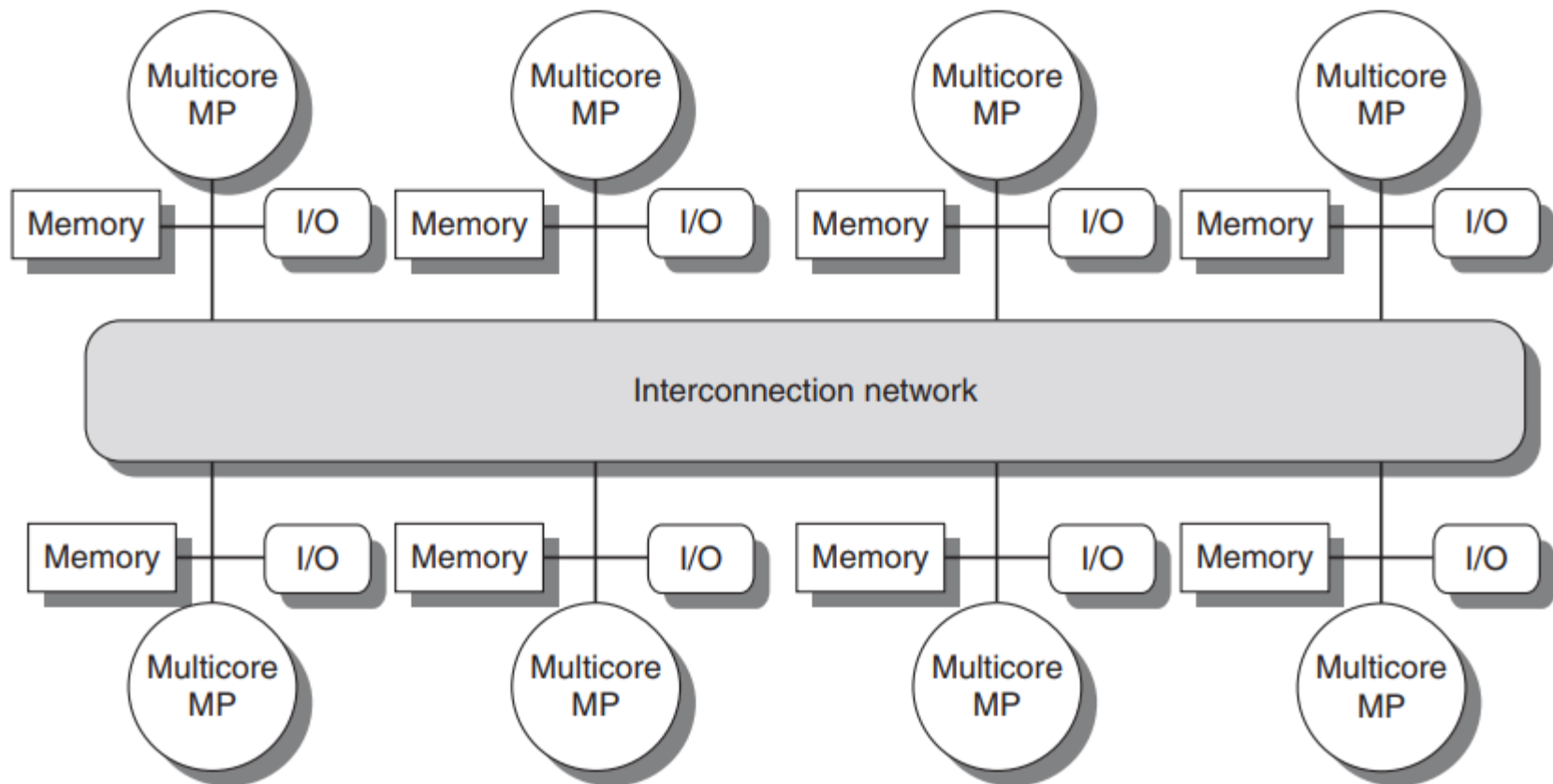
Decentralized Memory versions

- 1. Shared Memory with "Non Uniform Memory Access" time (NUMA)**
- 2. Message passing "multicomputer" with separate address space per processor**
 - Can invoke software with Remote Procedure Call (RPC)
 - Often via library, such as MPI: Message Passing Interface
 - Also called "Synchronous communication" since communication causes synchronization between 2 processes

Shared-Memory Multiprocessors(SMP)



Distributed-Memory Multiprocessors (DSM)



Parallel Architecture

- **Parallel Architecture extends traditional computer architecture with a communication architecture**
 - abstractions (HW/SW interface)
 - organizational structure to realize abstraction efficiently

Communication models

- **Shared Memory**

- Processors communicate with shared address space
- Easy on small-scale machines
- Advantages:
 - » Model of choice for uniprocessors, small-scale MPs
 - » Ease of programming
 - » Lower latency
 - » Easier to use hardware controlled caching

- **Message passing**

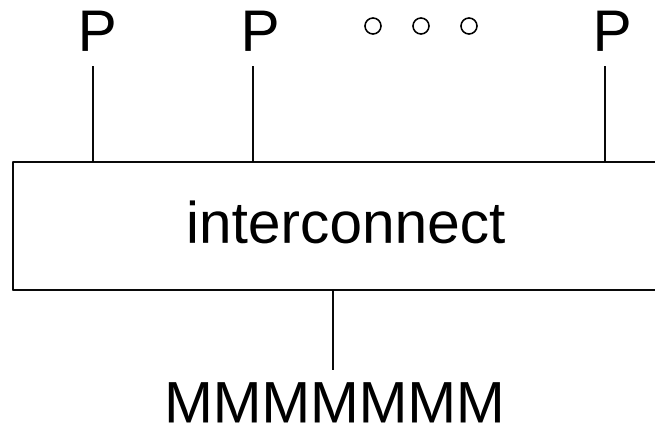
- Processors have private memories, communicate via messages
- Advantages:
 - » Less hardware, easier to design
 - » Focuses attention on costly **non-local** operations

- **Can support either SW model on either HW base**

Advantages Shared-Memory Communication Model

- Compatibility with SMP hardware
- Ease of programming when communication patterns are complex or vary dynamically during execution
- Ability to develop apps using familiar SMP model, attention only on performance critical accesses
- Lower communication overhead, better use of BW for small items, due to implicit communication and memory mapping to implement protection in hardware, rather than through I/O system
- HW-controlled caching to reduce remote comm. by caching of all data, both shared and private.

Communication Models: Shared-Memory



Each node a processor that runs a process

One shared memory

Accessible by any processor

The same address on two different processors refers to the same datum

Therefore, write and read memory to

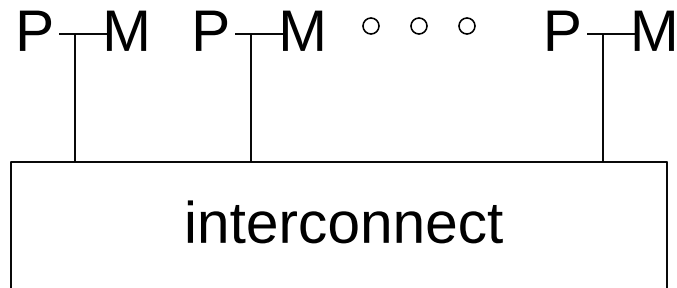
Store and recall data

Communicate, Synchronize (coordinate)

Advantages Message-Passing Communication Model

- The hardware can be simpler (esp. vs. NUMA)
- Communication explicit => simpler to understand; in shared memory it can be hard to know when communicating and when not, and how costly it is
- Explicit communication focuses attention on costly aspect of parallel computation, sometimes leading to improved structure in multiprocessor program
- Synchronization is naturally associated with sending messages, reducing the possibility for errors introduced by incorrect synchronization
- Easier to use sender-initiated communication, which may have some advantages in performance

Communication Models: Message Passing



Each node a computer

- Processor – runs its own program (like SM)

- Memory – local to that node, unrelated to other memory

Add messages for internode communication, send and receive like mail

Architectures

All mechanisms can usually be synthesized by all hardware

Key: which communication model does hardware support best?

Virtually all small-scale systems, multicores are shared-memory

Which is Best Communication Model to Support?

Shared-memory

- Used in small-scale systems

- Easier to program for dynamic data structures

- Lower overhead communication for small data

- Implicit movement of data with caching

- Hard to build?

Message-passing

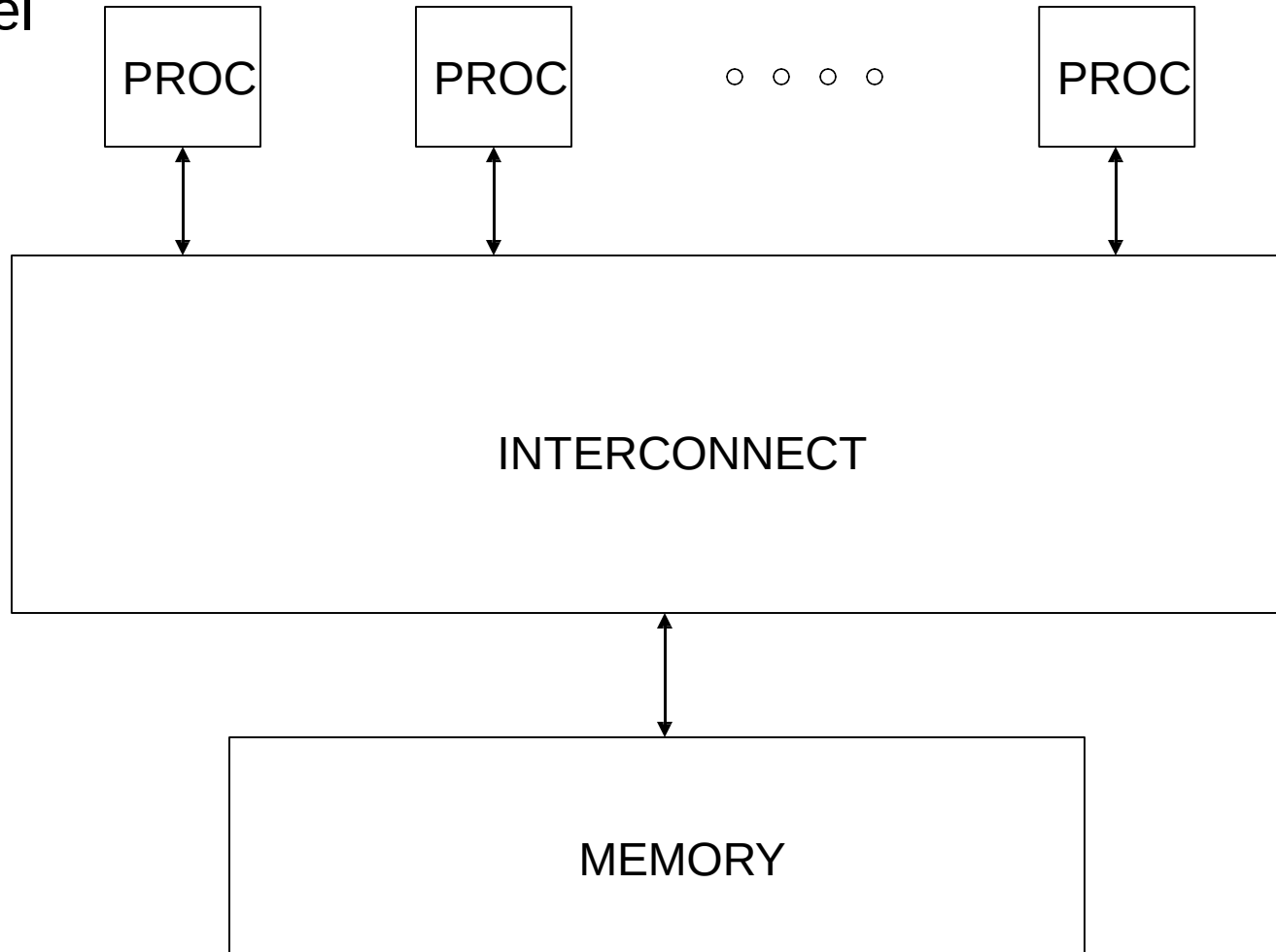
- Communication explicit harder to program?

- Larger overheads in communication OS intervention?

- Easier to build?

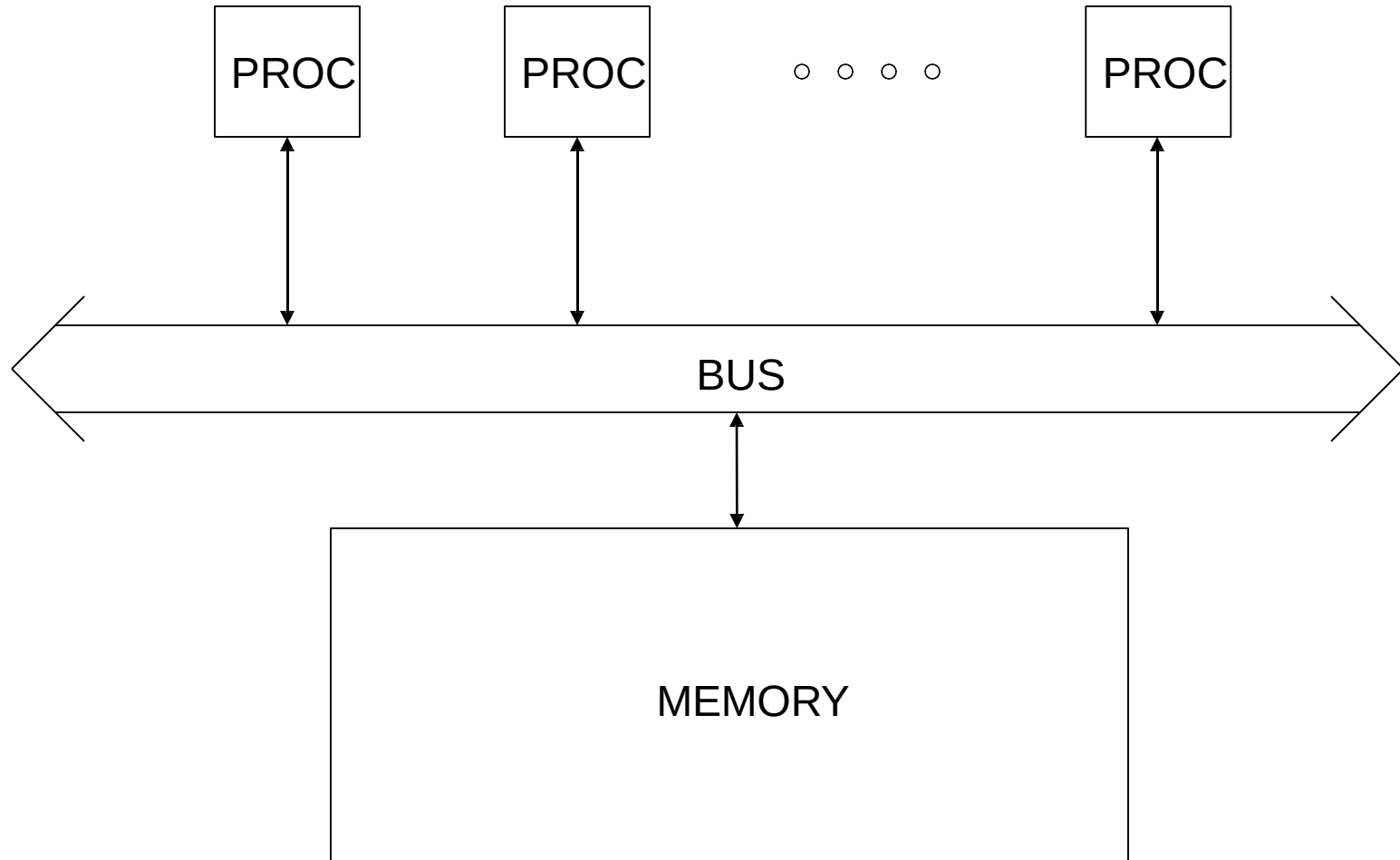
Shared-Memory Architecture

The model



For now, assume interconnect is a bus – *centralized architecture*

Centralized Shared-Memory Architecture



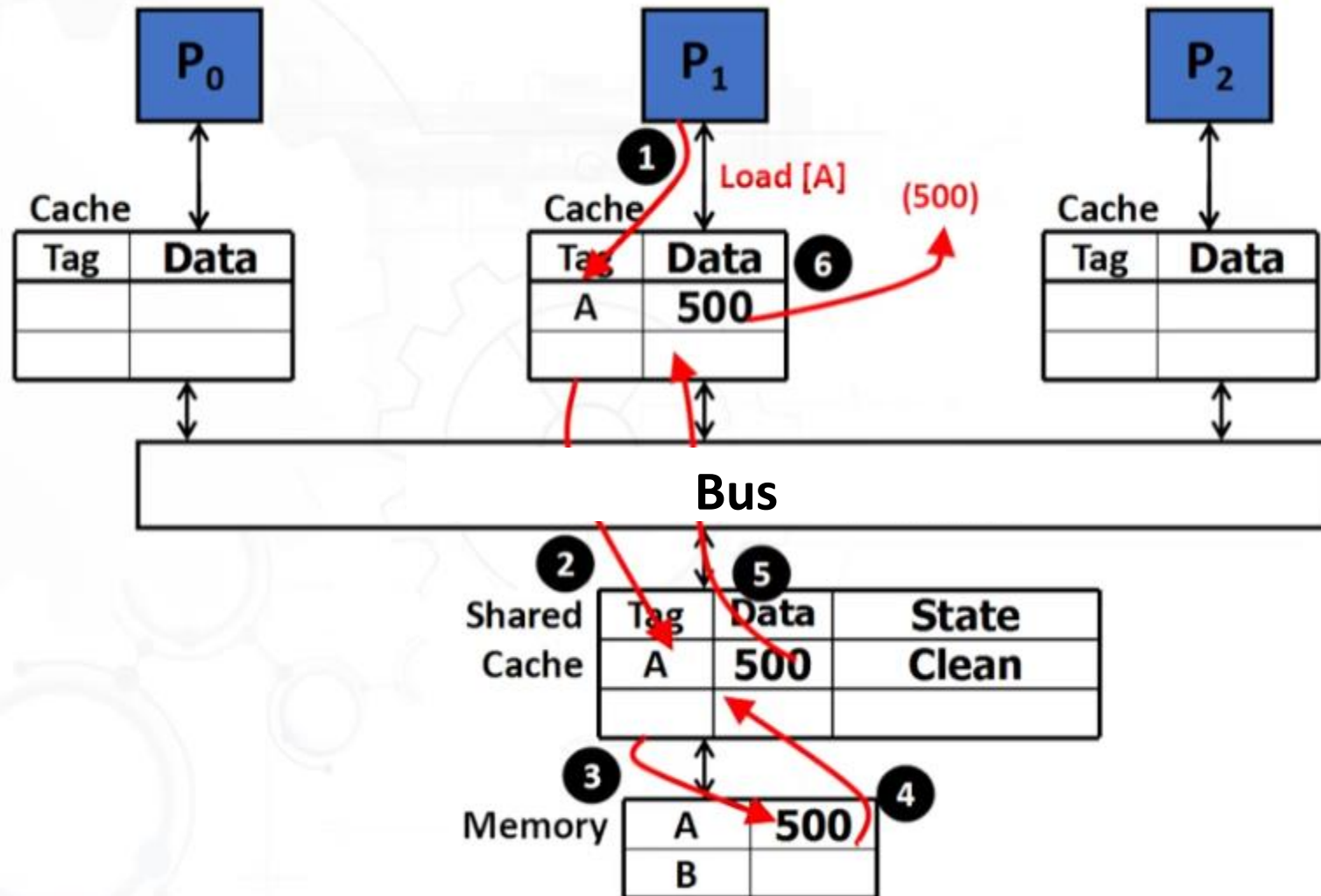
Centralized Shared-Memory Architecture (Cont.)

For higher bandwidth (throughput)

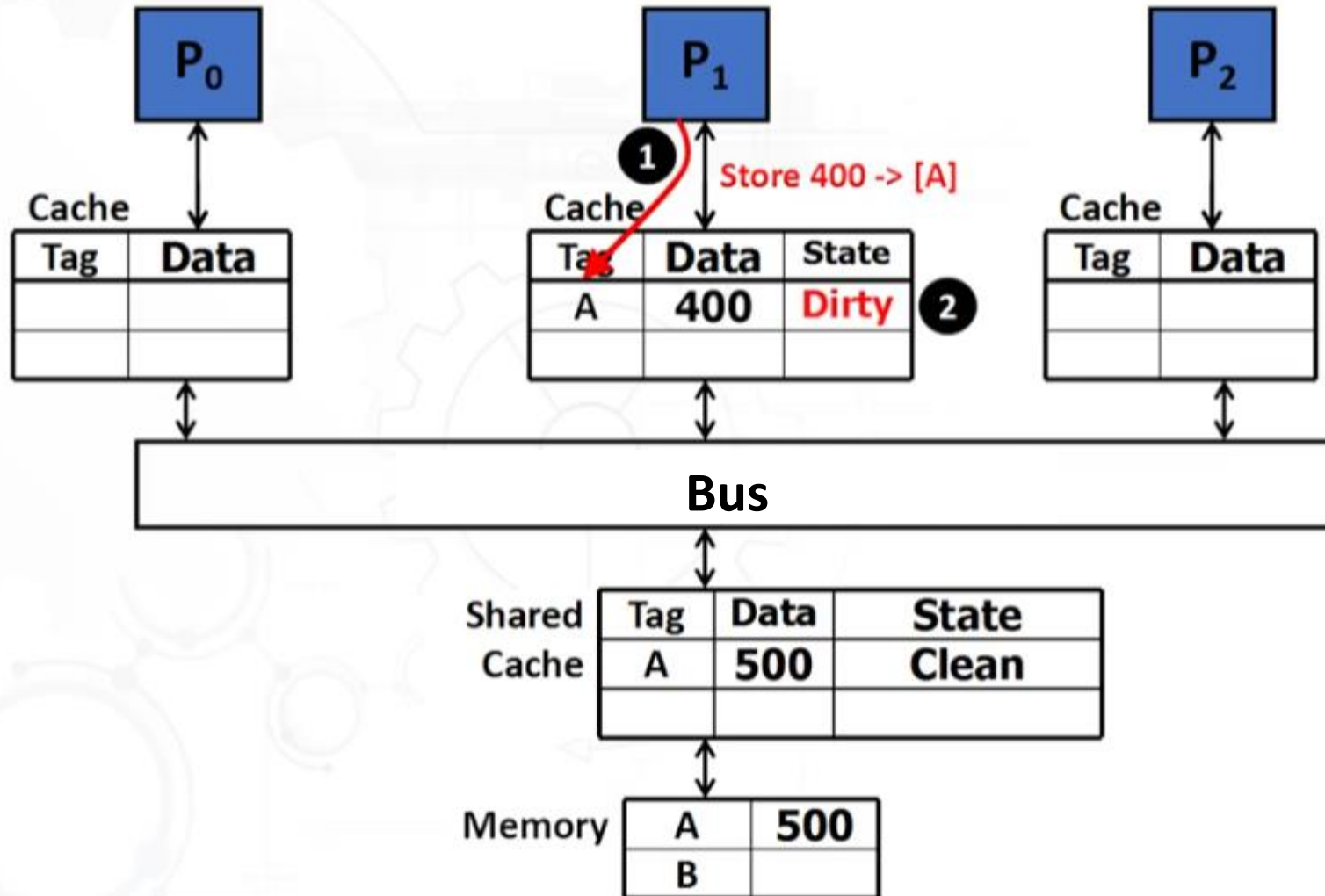
For lower latency

Problem?

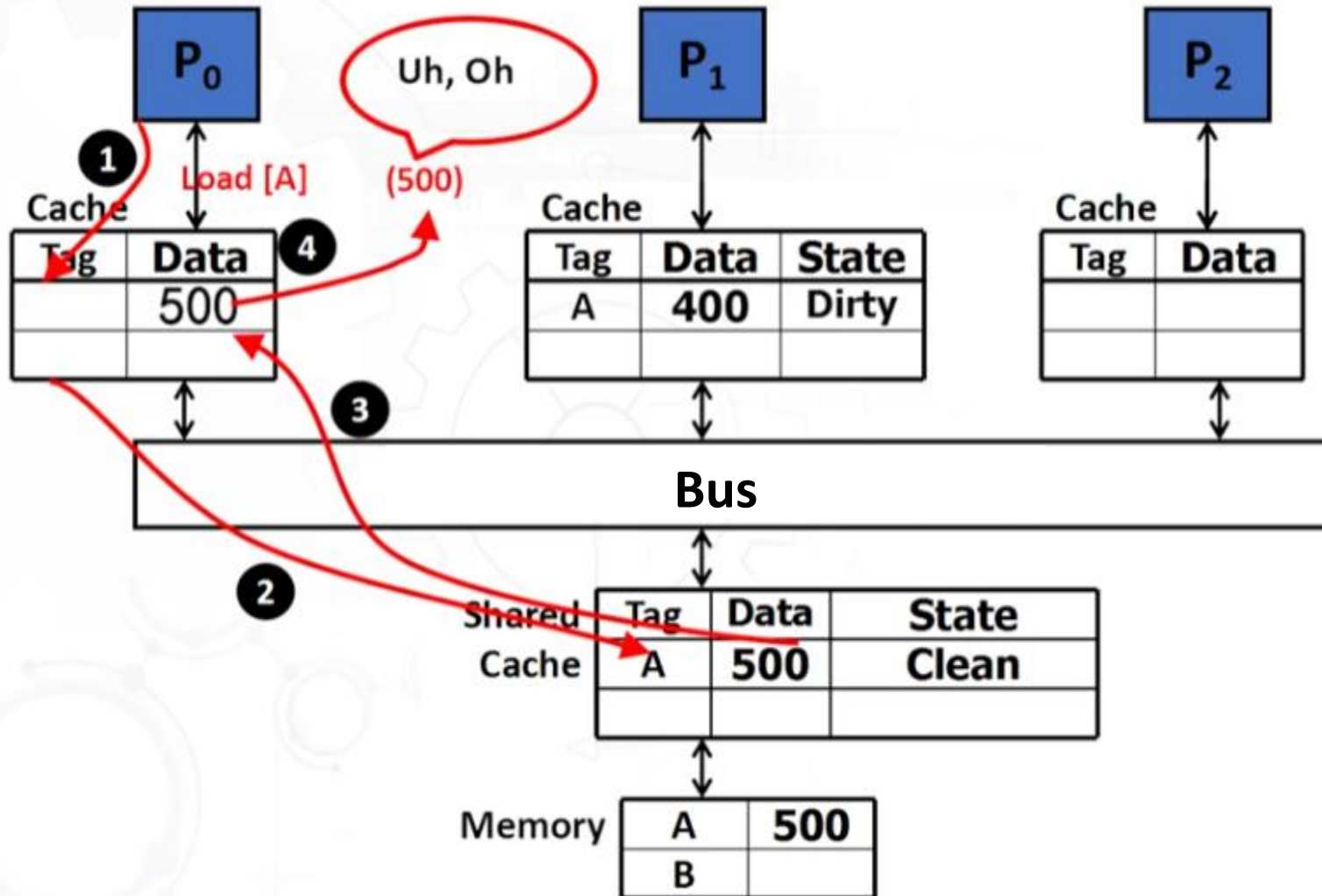
Cache Coherence Problem



Cache Coherence Problem



Cache Coherence Problem



What Does Coherency Mean?

- **Informally:**
 - "Any read must return the most recent write"
 - Too strict and too difficult to implement
- **Better:**
 - "Any write must eventually be seen by a read"
 - All writes are seen in proper order ("serialization")
- **Two rules to ensure this:**
 - "If P writes x and P1 reads it, P's write will be seen by P1 if the read and write are sufficiently far apart"
 - Writes to a single location are serialized:
seen in one order
 - » Latest write will be seen
 - » Otherwise could see writes in illogical order
(could see older value after a newer value)

Potential HW Coherency Solutions

- **Snooping Solution (Snoopy Bus):**
 - Send all requests for data to all processors
 - Processors snoop to see if they have a copy and respond accordingly
 - Requires broadcast, since caching information is at processors
 - Works well with bus (natural broadcast medium)
 - Dominates for small scale machines (most of the market)
- **Directory-Based Schemes (discuss later)**
 - Keep track of what is being shared in 1 centralized place (logically)
 - Distributed memory => distributed directory for scalability (avoids bottlenecks)
 - Send point-to-point requests to processors via network
 - Scales better than Snooping
 - Actually existed BEFORE Snooping-based schemes

Basic Snoopy Protocols

- Write **Invalidate** Protocol:

- Multiple readers, single writer
- Write to shared data: an invalidate is sent to all caches which snoop and **invalidate** any copies
- Read Miss:
 - » Write-through: memory is always up-to-date
 - » Write-back: snoop in caches to find most recent copy

- Write **Broadcast** Protocol (typically write through):

- Write to shared data: broadcast on bus, processors snoop, and **update** any copies
- Read miss: memory is always up-to-date

- **Write serialization**: **bus** serializes requests!

- Bus is single point of arbitration

Basic Snoopy Protocols

- **Write Invalidate versus Broadcast:**
 - Invalidate requires one transaction per write-run
 - Invalidate uses spatial locality: one transaction per block
 - Broadcast has lower latency between write and read

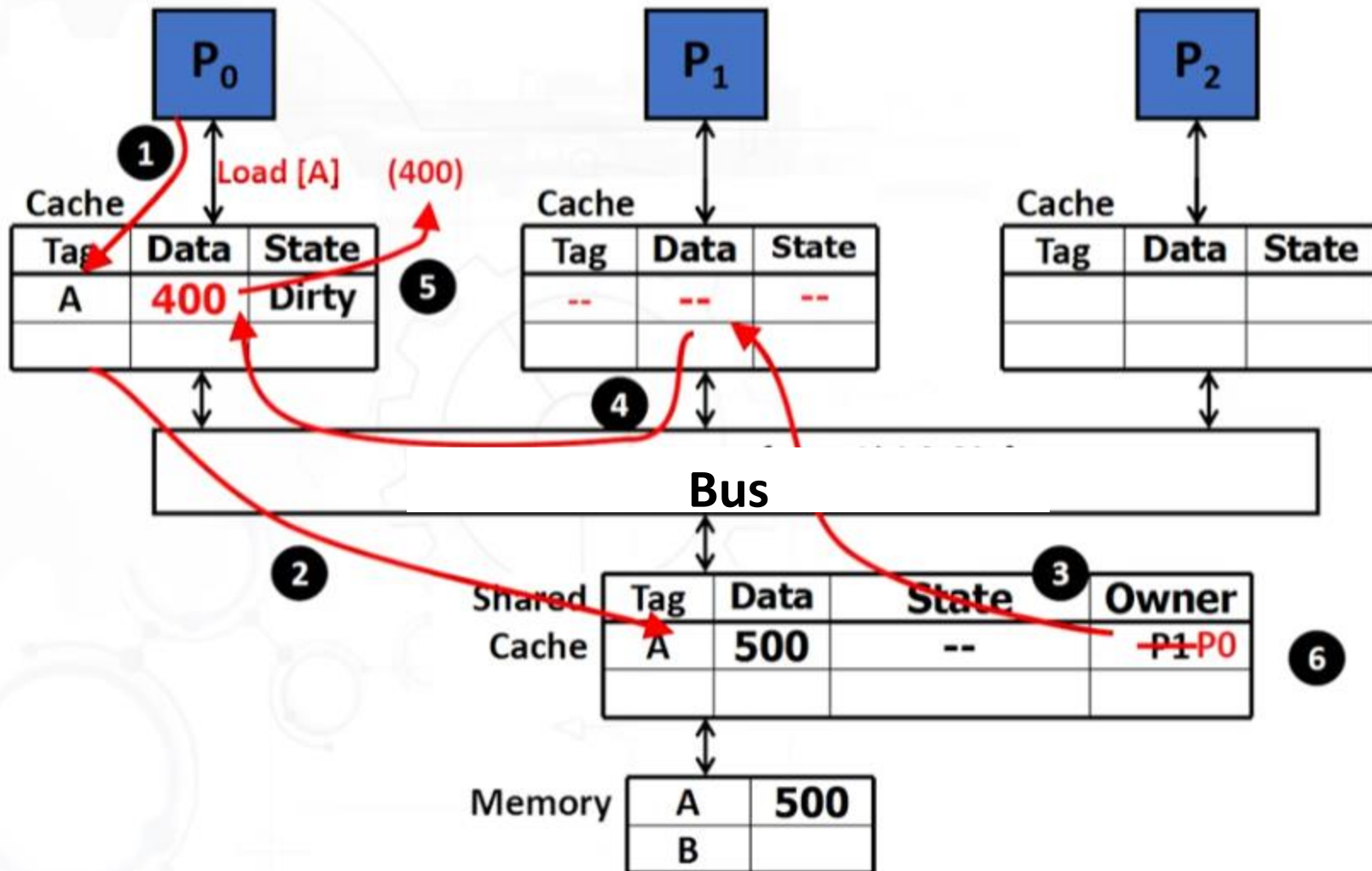
Write Invalidate Protocols

Processor activity	Bus activity	Contents of processor A's cache	Contents of processor B's cache	Contents of memory location X
				0
Processor A reads X	Cache miss for X	0		0
Processor B reads X	Cache miss for X	0	0	0
Processor A writes a 1 to X	Invalidation for X	1		0
Processor B reads X	Cache miss for X	1	1	1

Figure 5.4 An example of an invalidation protocol working on a snooping bus for a single cache block (X) with write-back caches. We assume that neither cache initially holds X and that the value of X in memory is 0. The processor and memory contents show the value after the processor and bus activity have both completed. A blank indicates no activity or no copy cached. When the second miss by B occurs, processor A responds with the value canceling the response from memory. In addition, both the contents of B's cache and the memory contents of X are updated. This update of memory, which occurs when a block becomes shared, simplifies the protocol, but it is possible to track the ownership and force the write-back only if the block is replaced. This requires the introduction of an additional state called "owner," which indicates that a block may be shared, but the owning processor is responsible for updating any other processors and memory when it changes the block or replaces it. If a multicore uses a shared cache (e.g., L3), then all memory is seen through the shared cache; L3 acts like the memory in this example, and coherency must be handled for the private L1 and L2 for each core. It is this observation that led some designers to opt for a directory protocol within the multicore. To make this work the L3 cache must be inclusive (see page 397).

Cache Coherence Solutions

Snooping



Problem with centralized architecture

MSI Coherence Protocol

- **Three-state protocol, MSI**
- **Each cache block is in one state :**
 - Modified (M) : The block has been modified in the cache;
The block can be read and write;
 - Shared (S) : This block is unmodified and exists in read-only state in at least one cache.
 - Invalid (I) : This block is either not present in the current cache or has been invalidated by a bus request.

MSI Coherence Protocol

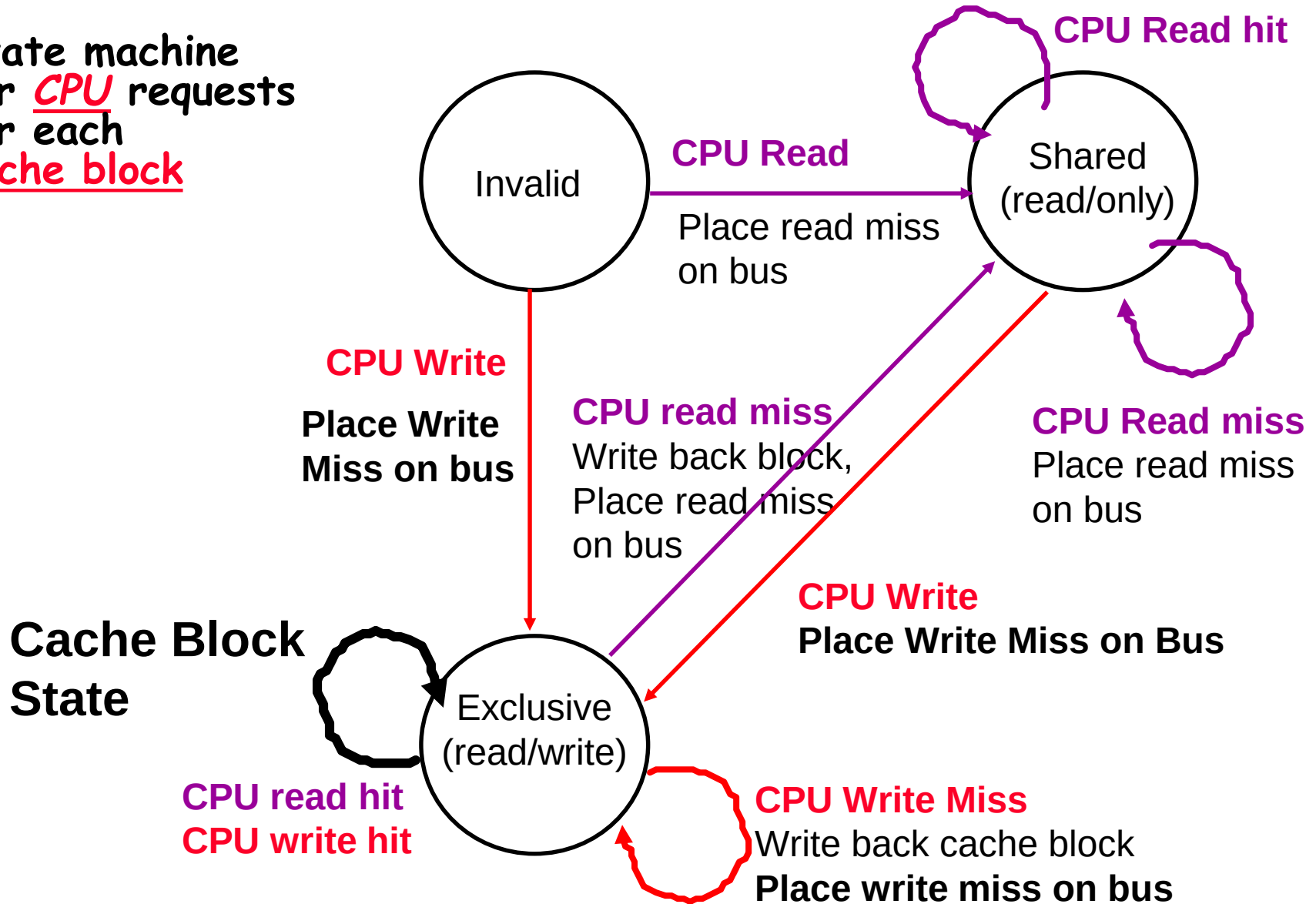
- **Invalidation protocol, write-back cache**
- **Each block of memory is in one state:**
 - Clean in all caches and up-to-date in memory (Shared)
 - OR Dirty in exactly one cache (Exclusive)
 - OR Not in any caches
- **Each cache block is in one state (track these):**
 - Shared : block can be read
 - OR Exclusive : cache has only copy, its writeable, and dirty
 - OR Invalid : block contains no data
- **Read misses: cause all caches to snoop bus**
- **Writes to clean line are treated as misses**

MSI Coherence Protocol

Request	Source	State of addressed cache block	Type of cache action	Function and explanation
Read hit	Processor	Shared or modified	Normal hit	Read data in local cache.
Read miss	Processor	Invalid	Normal miss	Place read miss on bus.
Read miss	Processor	Shared	Replacement	Address conflict miss: place read miss on bus.
Read miss	Processor	Modified	Replacement	Address conflict miss: write-back block, then place read miss on bus.
Write hit	Processor	Modified	Normal hit	Write data in local cache.
Write hit	Processor	Shared	Coherence	Place invalidate on bus. These operations are often called upgrade or <i>ownership</i> misses, since they do not fetch the data but only change the state.
Write miss	Processor	Invalid	Normal miss	Place write miss on bus.
Write miss	Processor	Shared	Replacement	Address conflict miss: place write miss on bus.
Write miss	Processor	Modified	Replacement	Address conflict miss: write-back block, then place write miss on bus.
Read miss	Bus	Shared	No action	Allow shared cache or memory to service read miss.
Read miss	Bus	Modified	Coherence	Attempt to share data: place cache block on bus and change state to shared.
Invalidate	Bus	Shared	Coherence	Attempt to write shared block; invalidate the block.
Write miss	Bus	Shared	Coherence	Attempt to write shared block; invalidate the cache block.
Write miss	Bus	Modified	Coherence	Attempt to write block that is exclusive elsewhere; write-back the cache block and make its state invalid in the local cache.

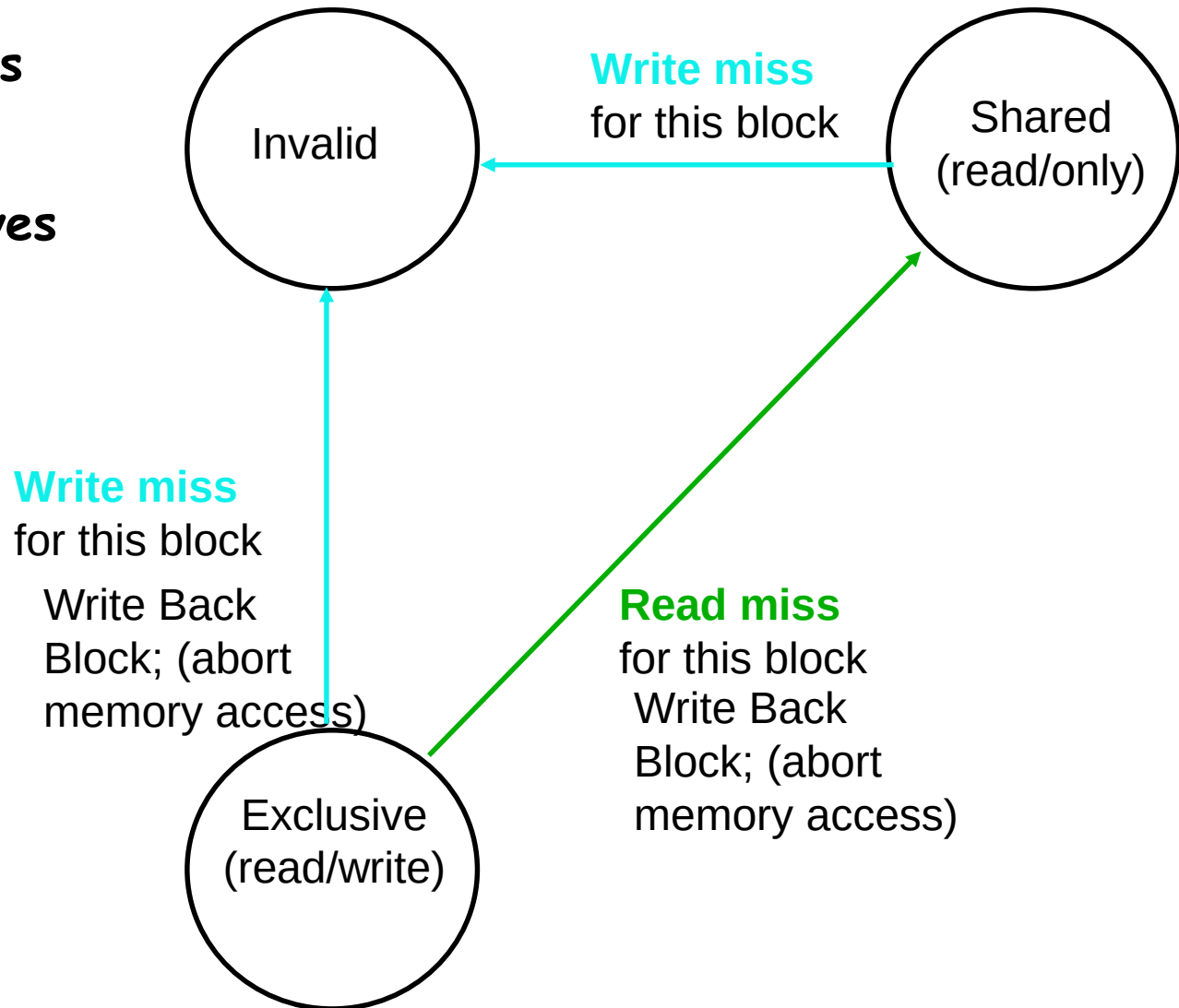
MSI Coherence Protocol

- State machine for CPU requests for each cache block



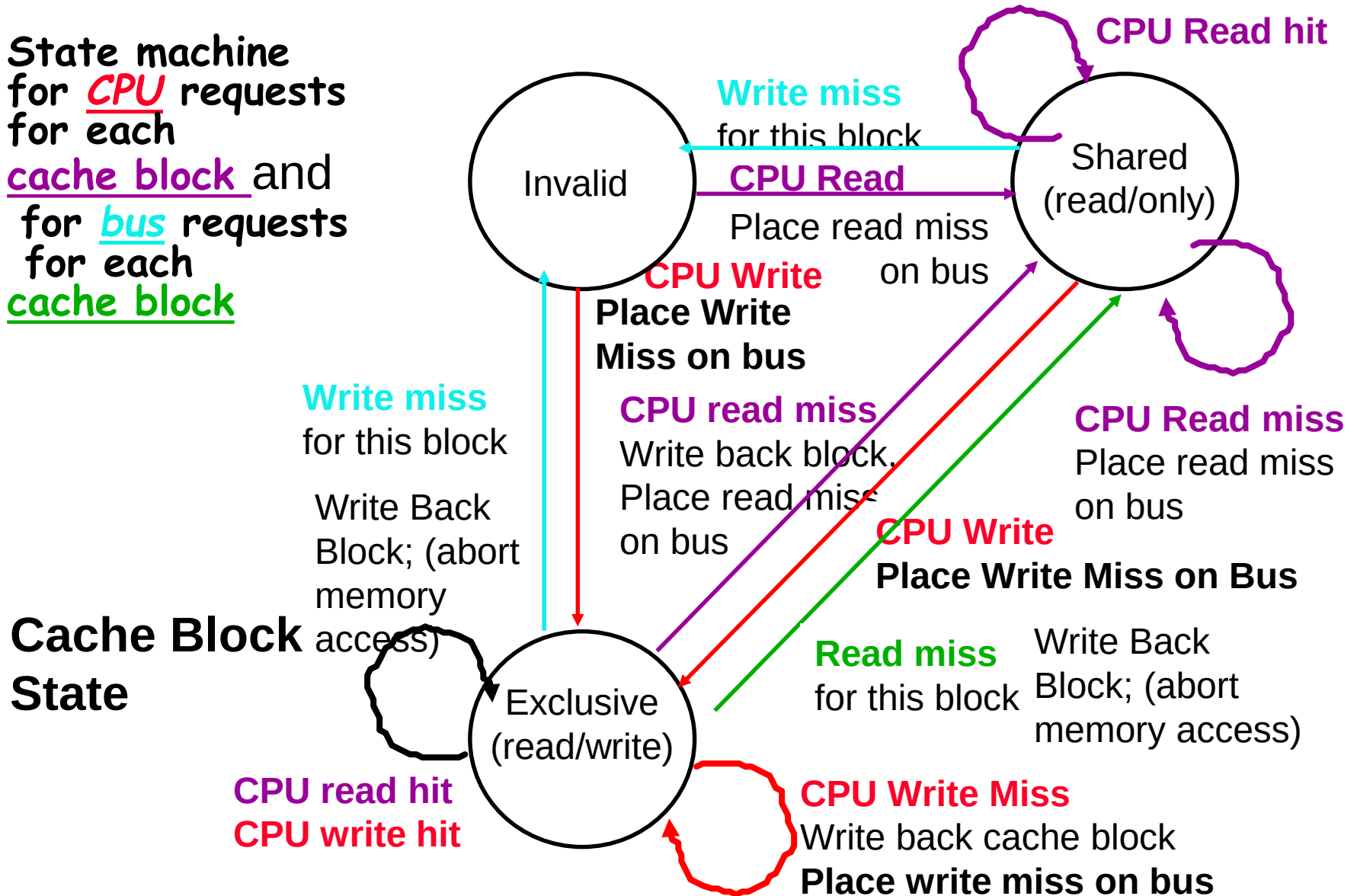
MSI Coherence Protocol

- State machine for bus requests for each cache block
- Appendix E? gives details of bus requests

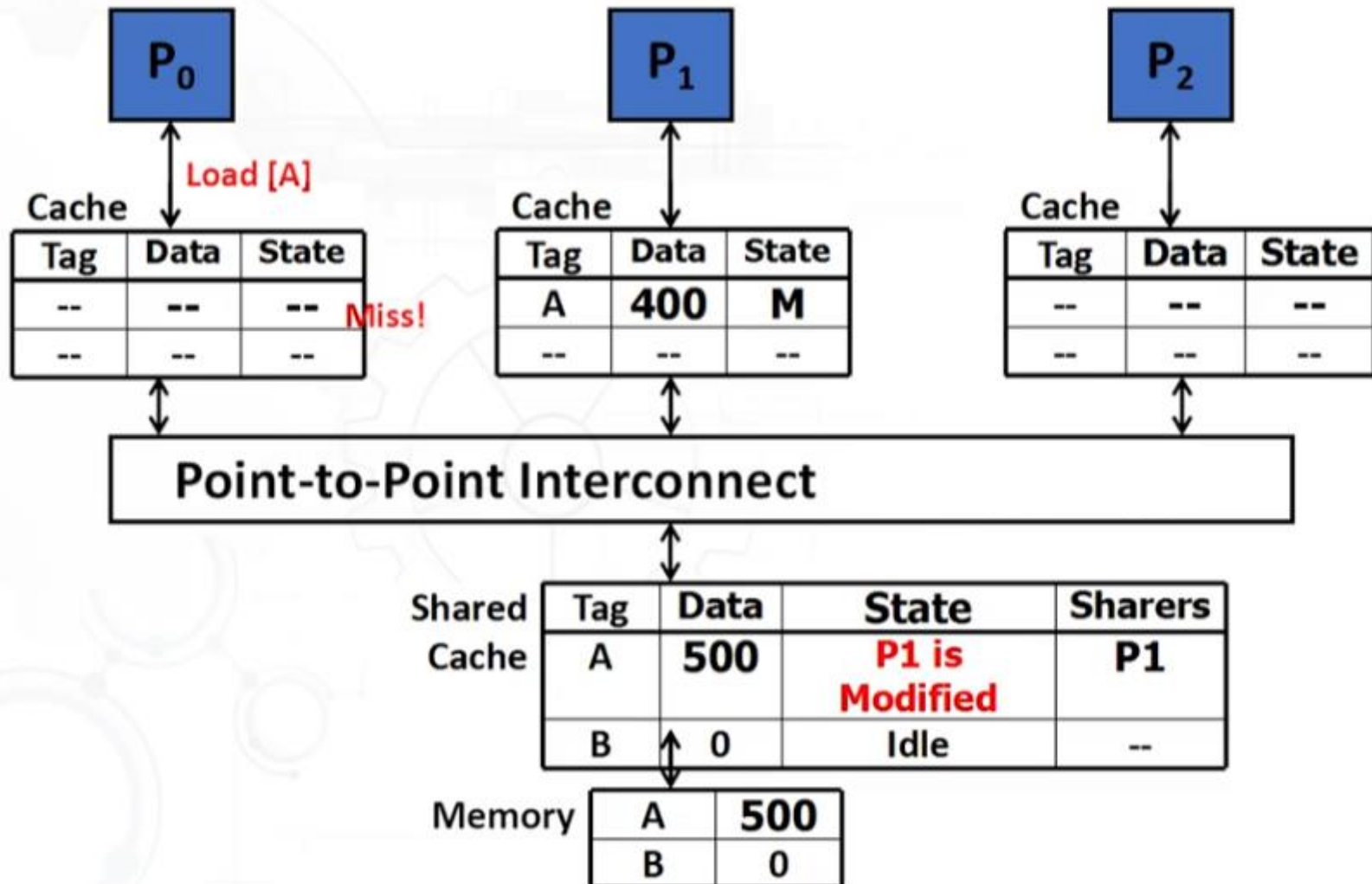


MSI Coherence Protocol

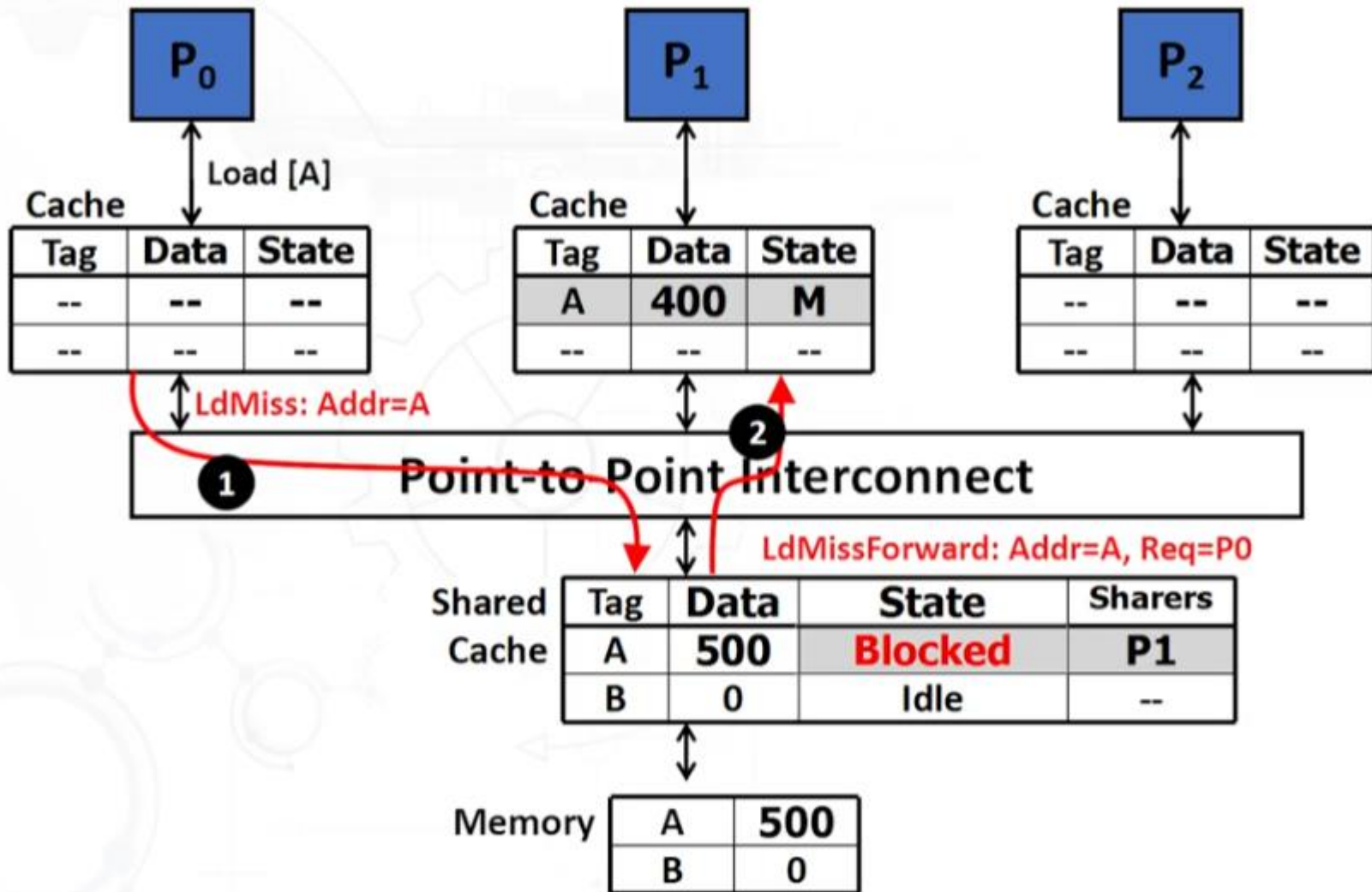
- State machine for **CPU** requests for each **cache block** and for **bus** requests for each **cache block**



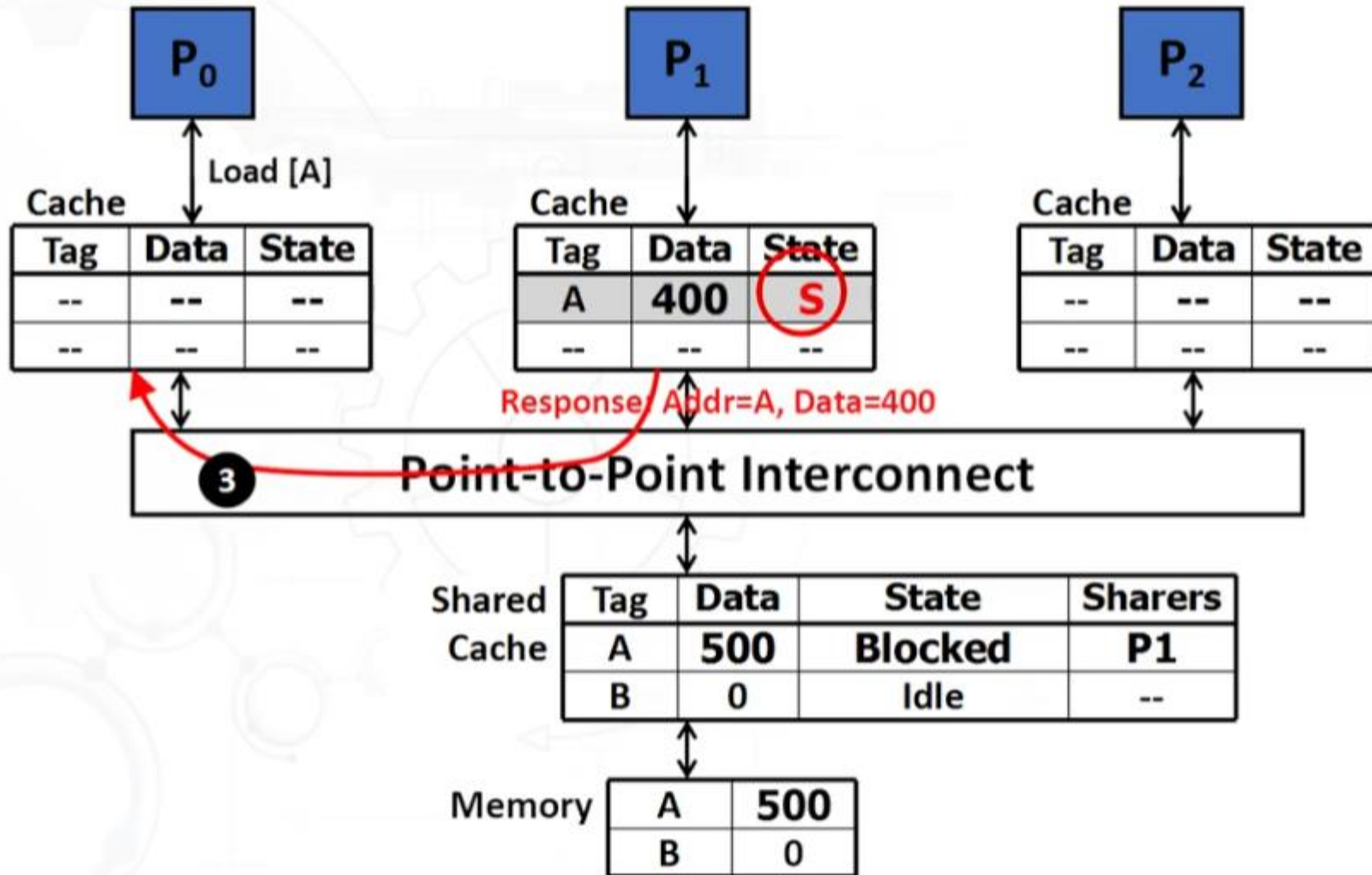
MSI Coherence Protocol Example: Step 1



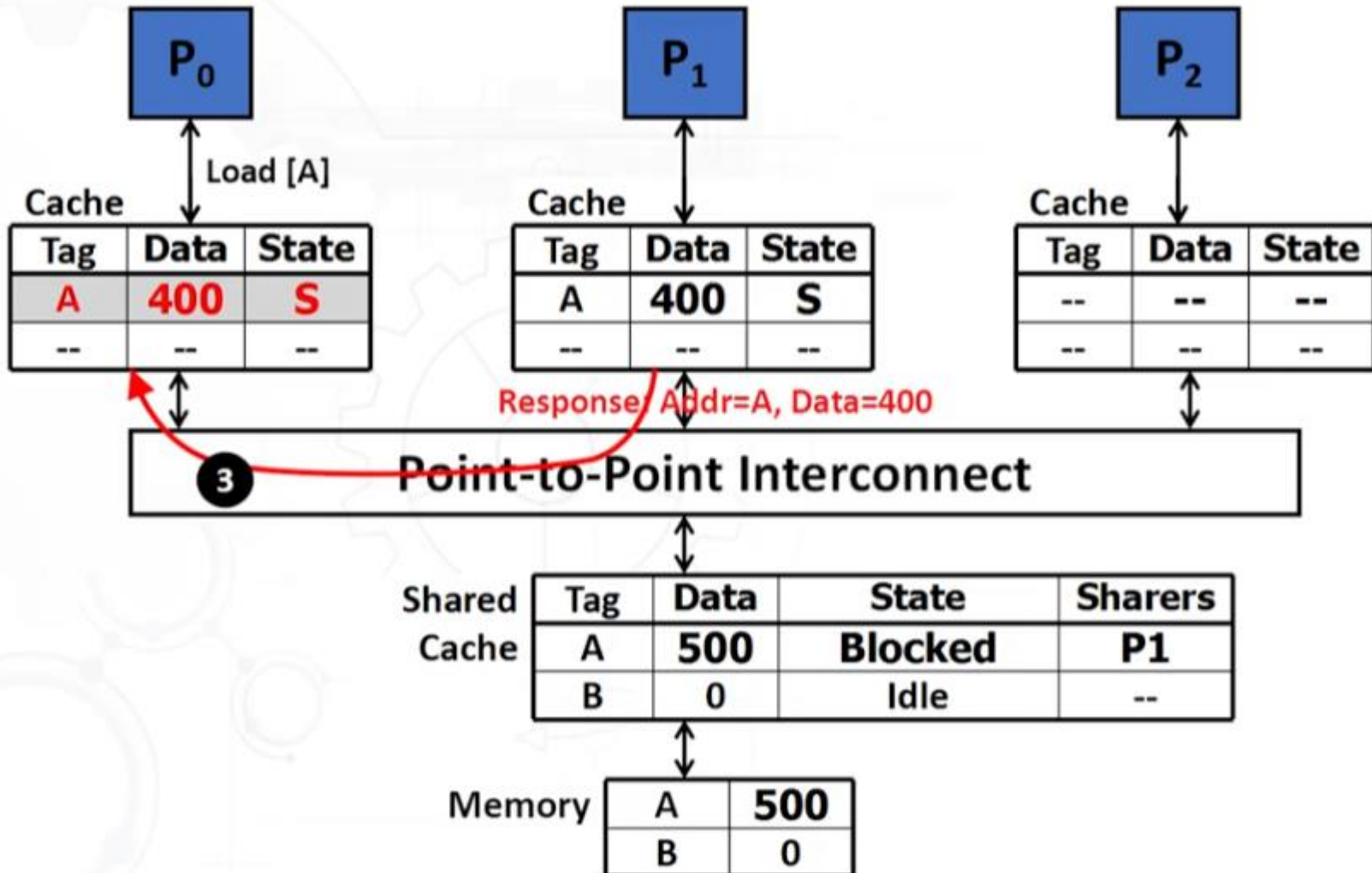
MSI Coherence Protocol Example: Step 2



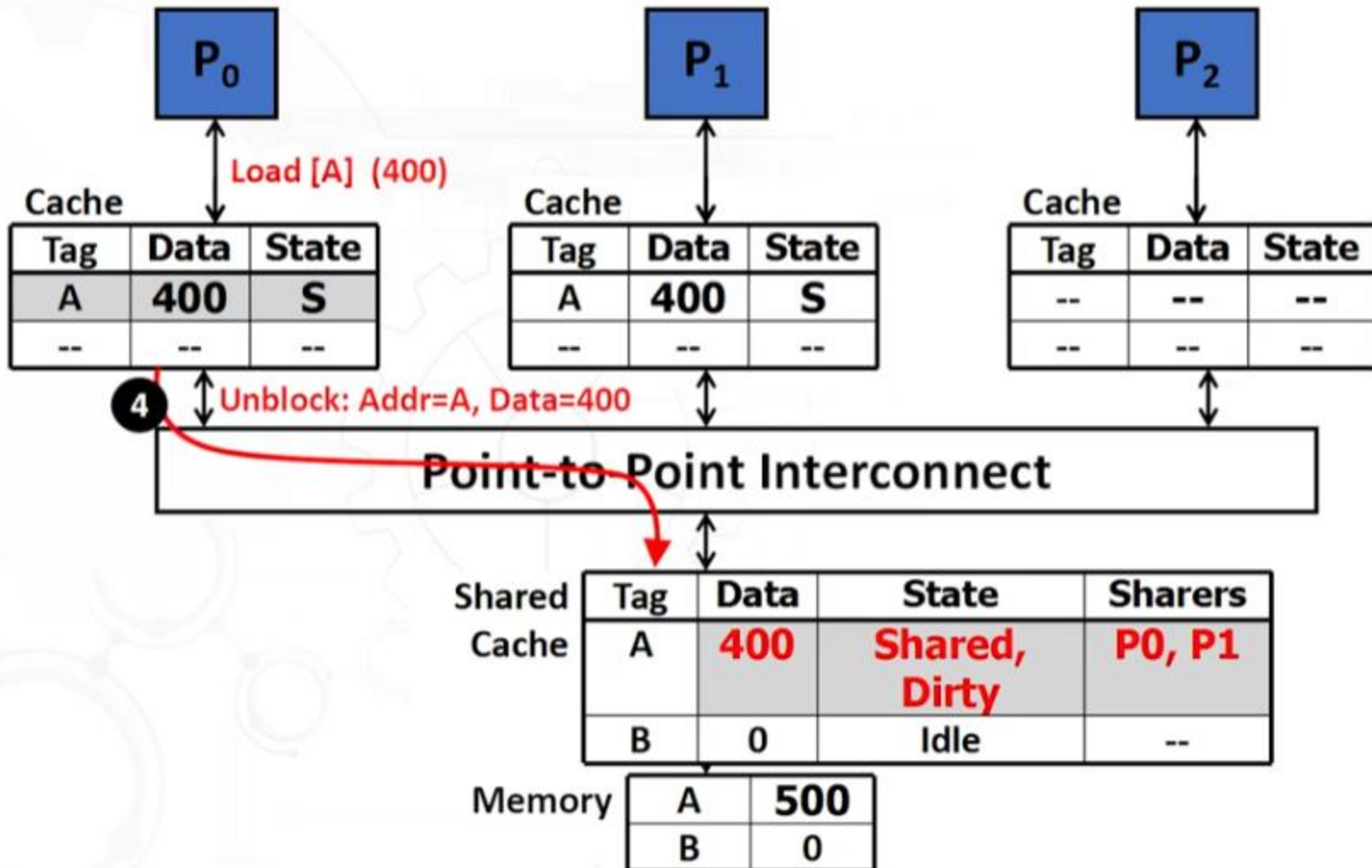
MSI Coherence Protocol Example: Step 3



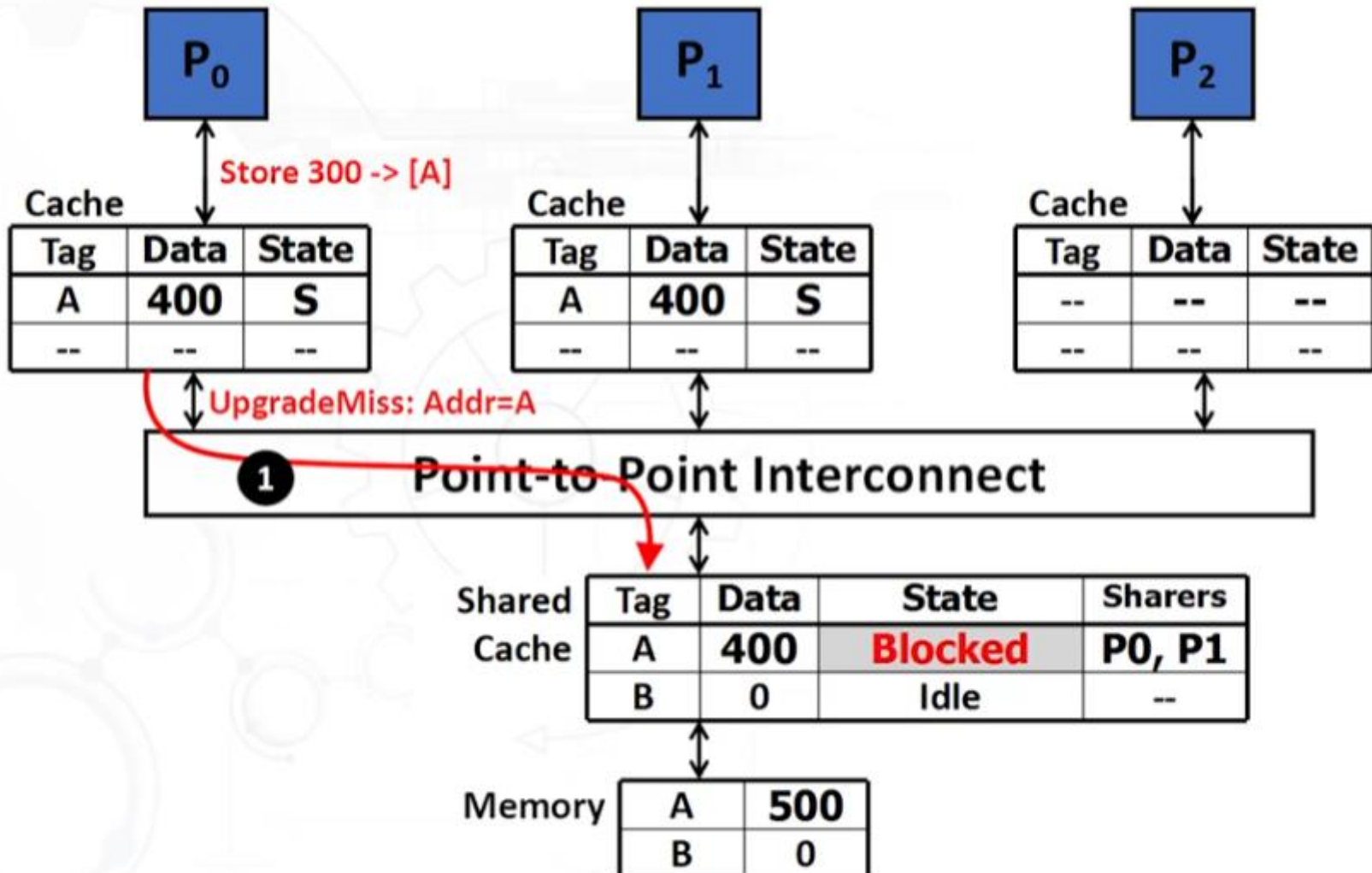
MSI Coherence Protocol Example: Step 4



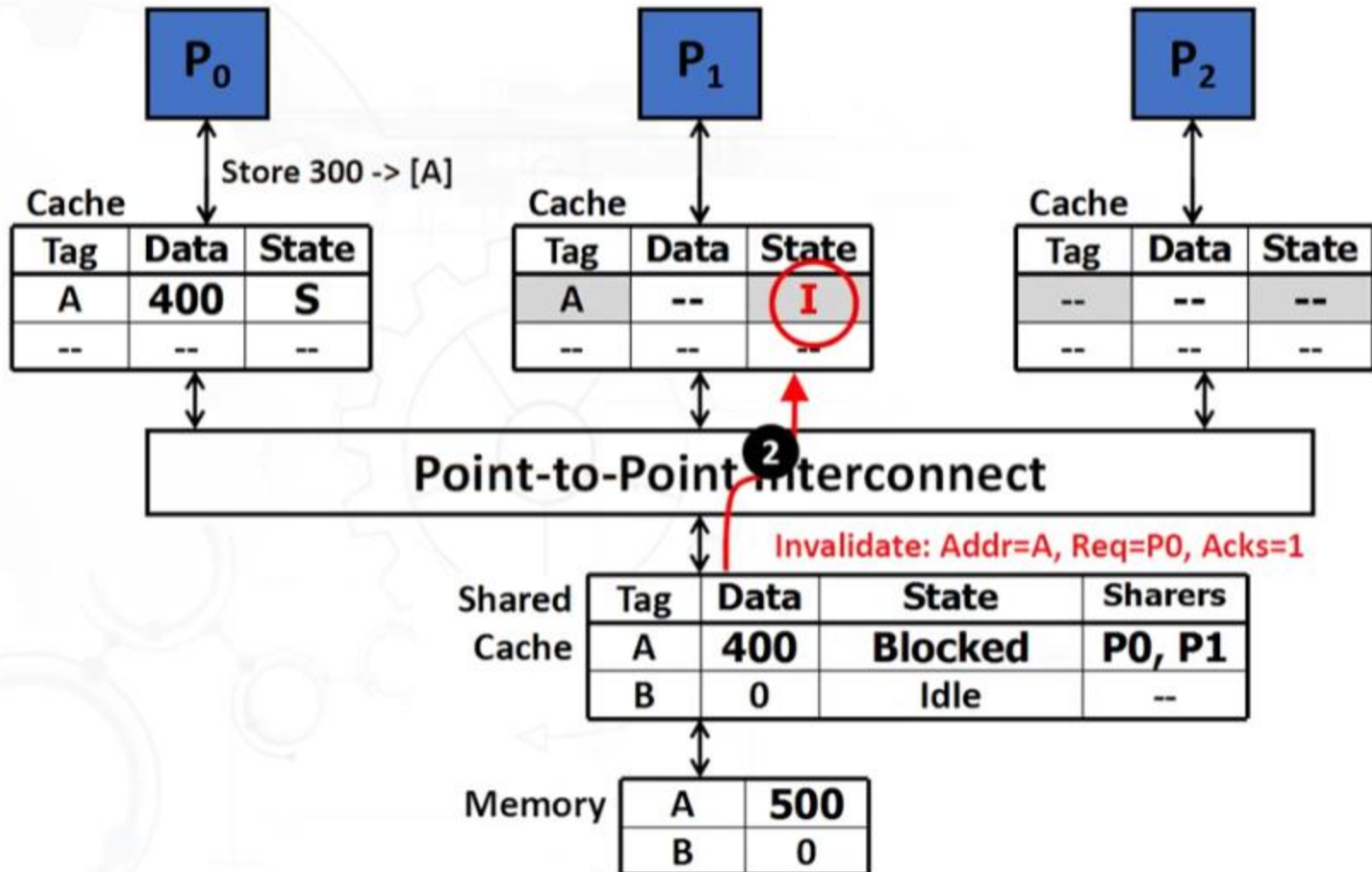
MSI Coherence Protocol Example: Step 5



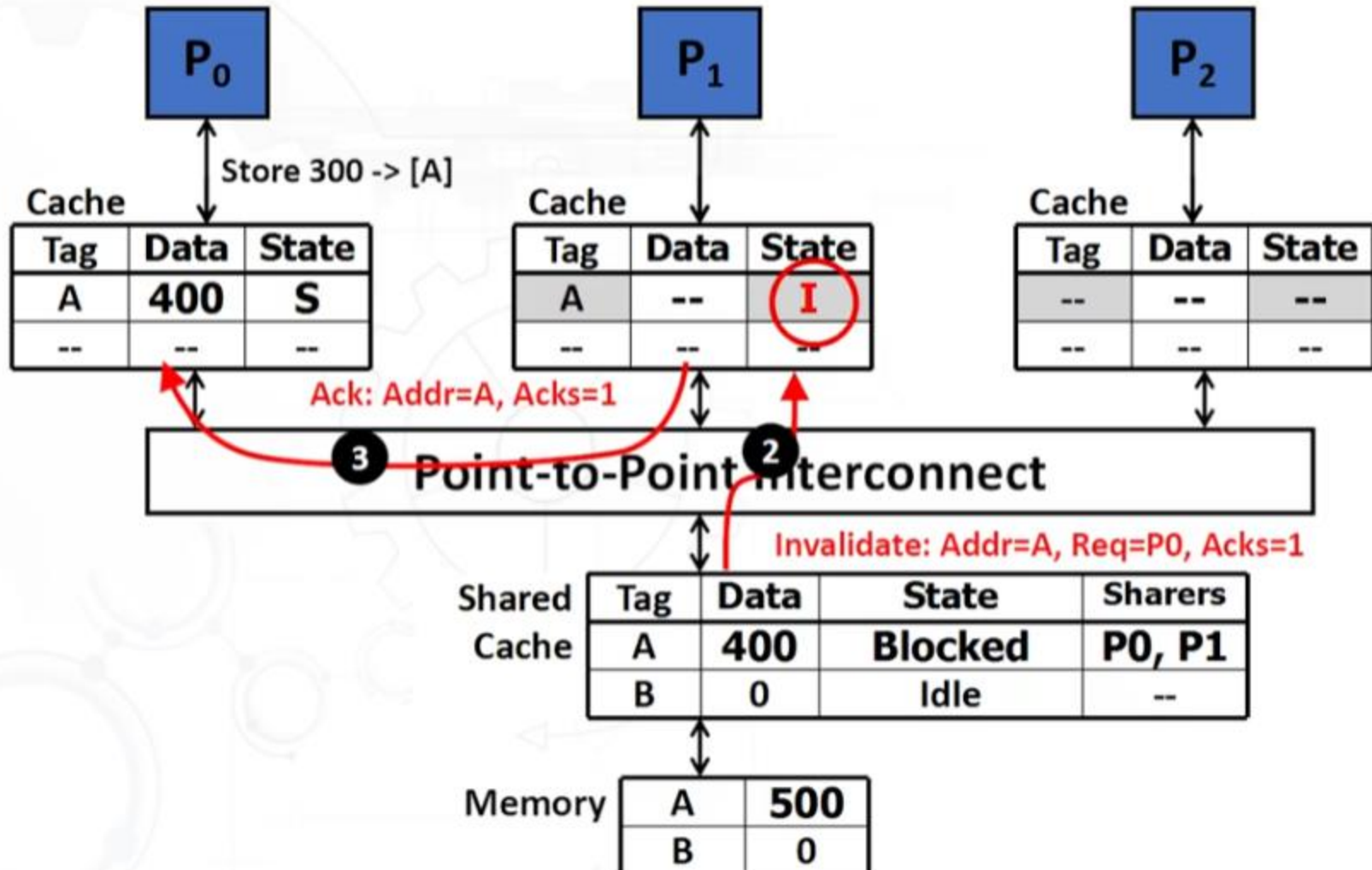
MSI Coherence Protocol Example: Step 6



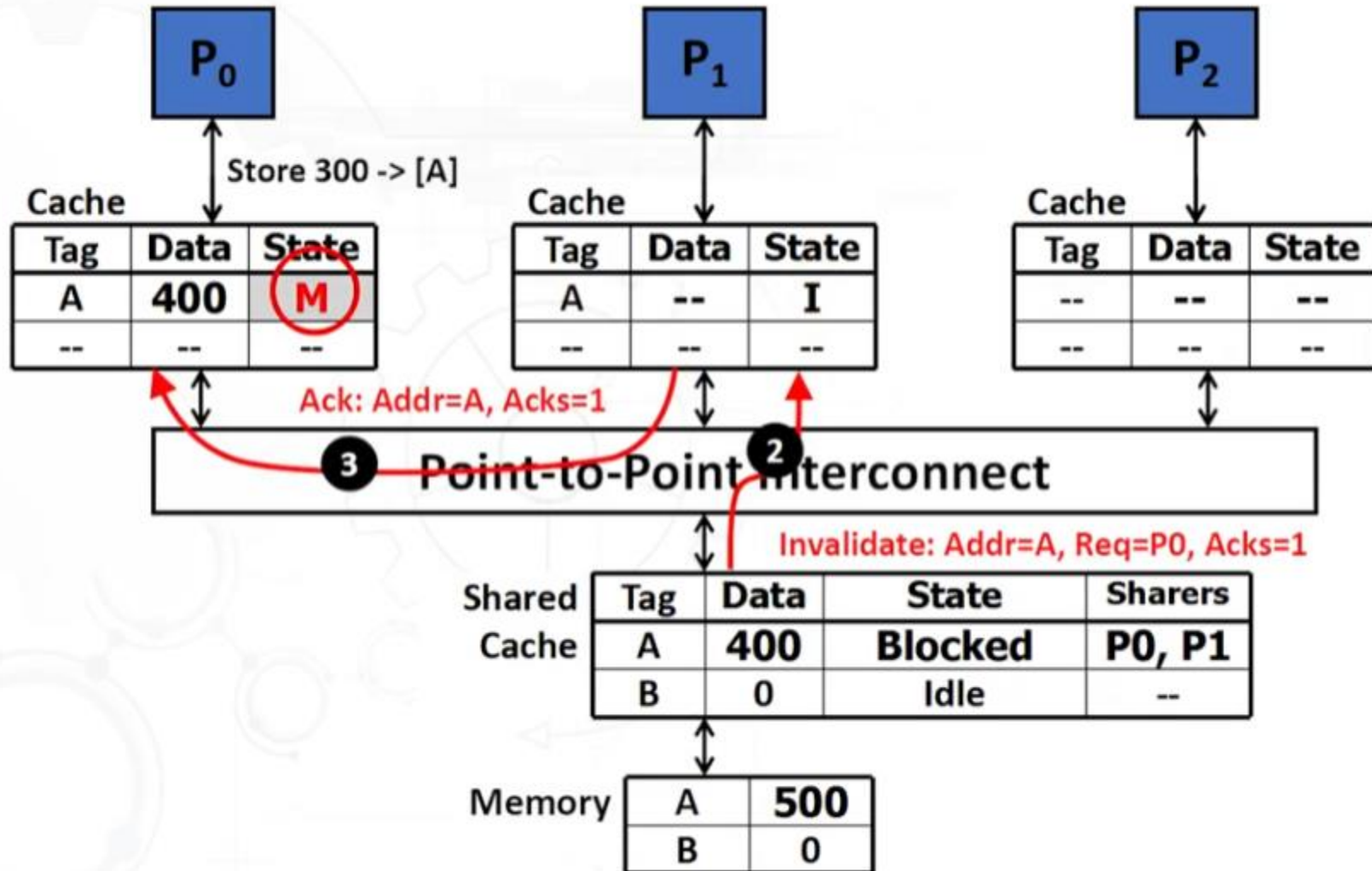
MSI Coherence Protocol Example: Step 7



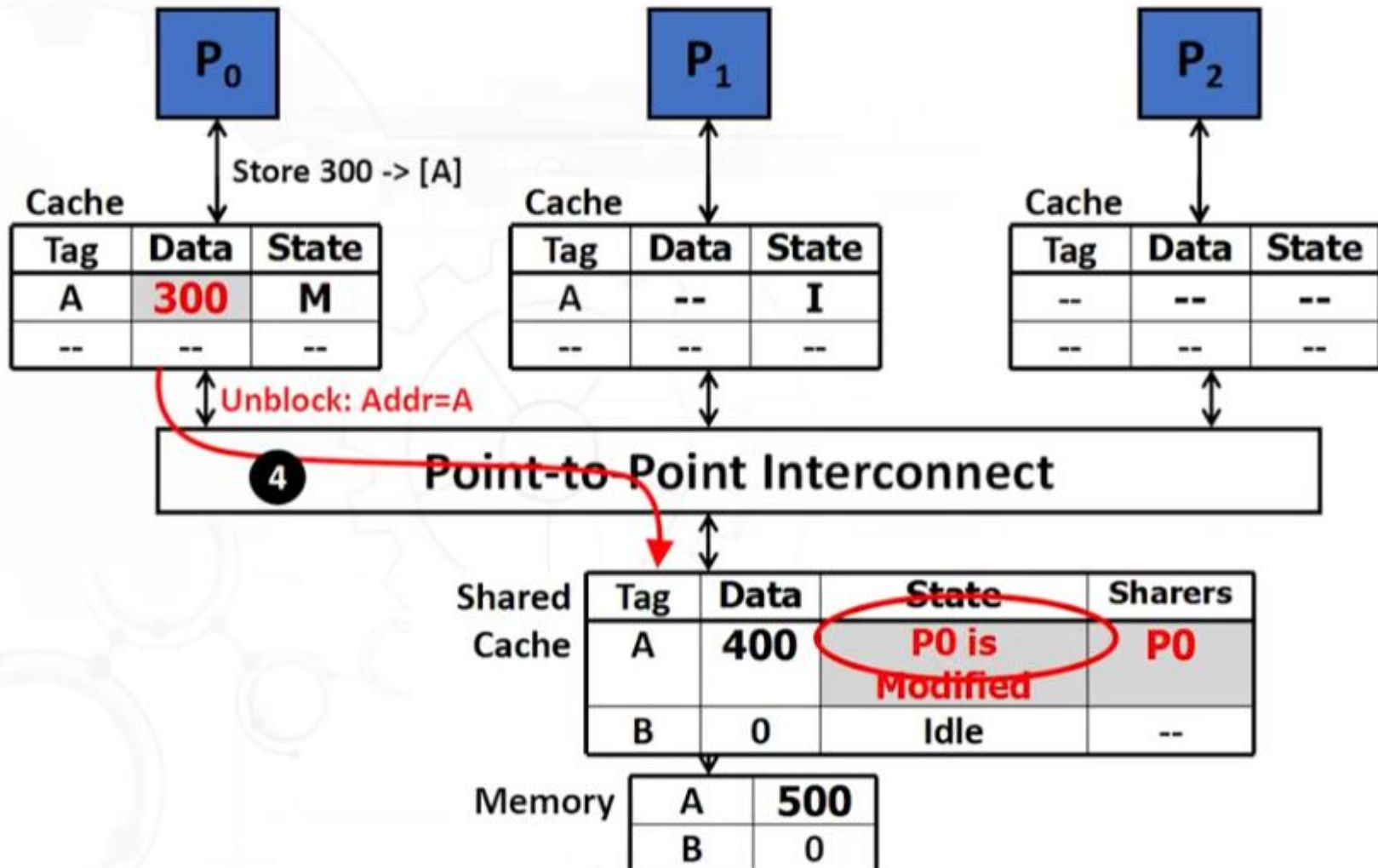
MSI Coherence Protocol Example: Step 8



MSI Coherence Protocol Example: Step 9



MSI Coherence Protocol Example: Step 10



Snooping Cache Variations

Basic Protocol	Berkeley Protocol	Illinois Protocol	MESI Protocol
Exclusive	Owned Exclusive	Private Dirty	<u>M</u> odified (private, !=Memory)
Shared	Owned Shared	Private Clean	e <u>X</u> clusive (private, =Memory)
Invalid	Shared	Shared	<u>S</u> hared (shared, =Memory)
	Invalid	Invalid	<u>I</u> nvalid

Owner can update via bus invalidate operation
 Owner must write back when replaced in cache

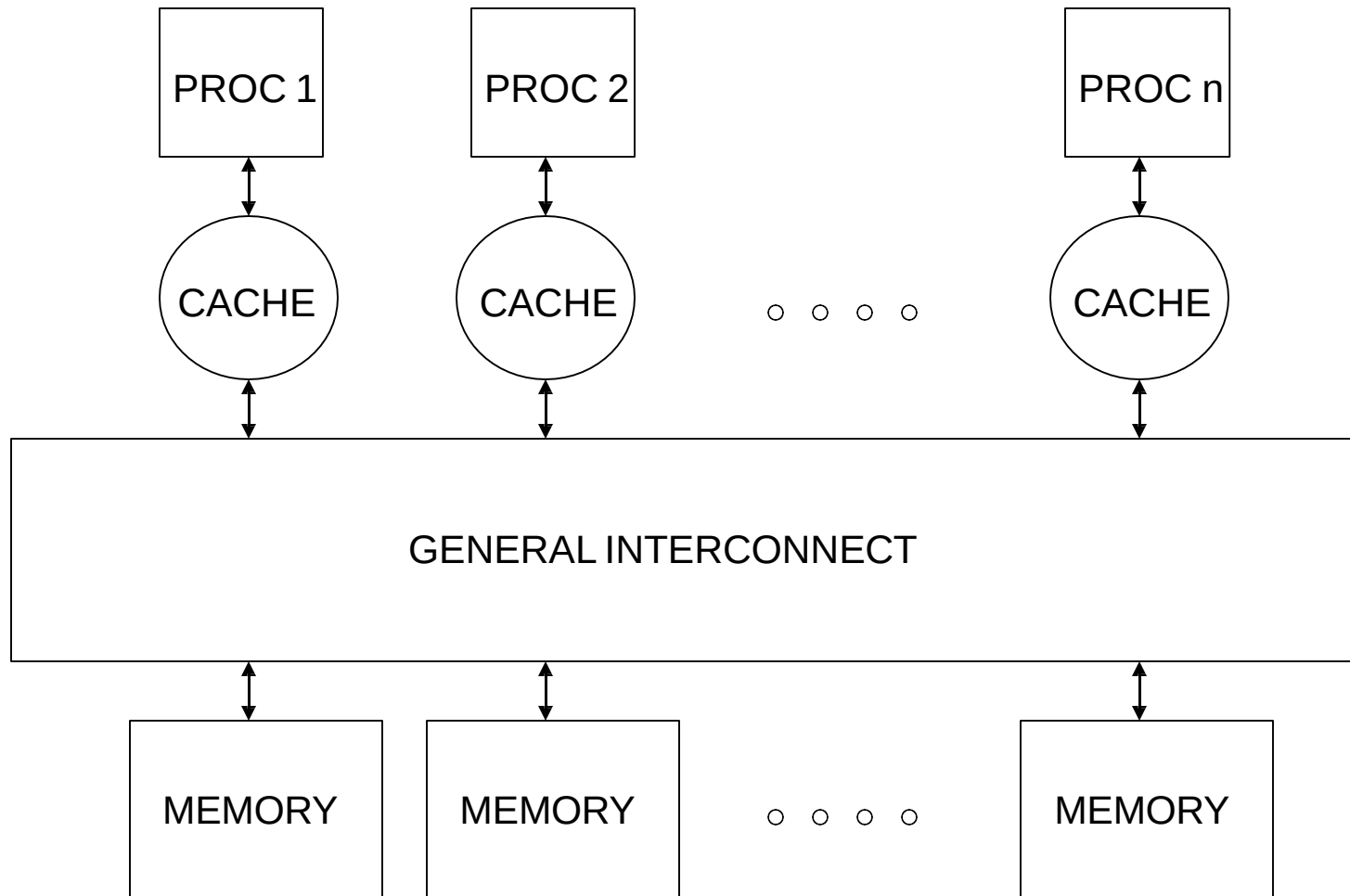
If read sourced from memory, then Private Clean
 if read sourced from other cache, then Shared
 Can write in cache if held private clean or dirty

Larger MPs

- **Separate Memory per Processor**
- **Local or Remote access via memory controller**
- **1 Cache Coherency solution: non-cached pages**
- **Alternative: directory per cache that tracks state of every block in every cache**
 - Which caches have a copies of block, dirty vs. clean, ...
- **Info per memory block vs. per cache block?**
 - PLUS: In memory => simpler protocol (centralized/one location)
 - MINUS: In memory => directory is $f(\text{memory size})$ vs. $f(\text{cache size})$
- **Prevent directory as bottleneck?**
distribute directory entries with memory, each keeping track of which Procs have copies of their blocks

Distributed Shared-Memory (DSM) Architecture

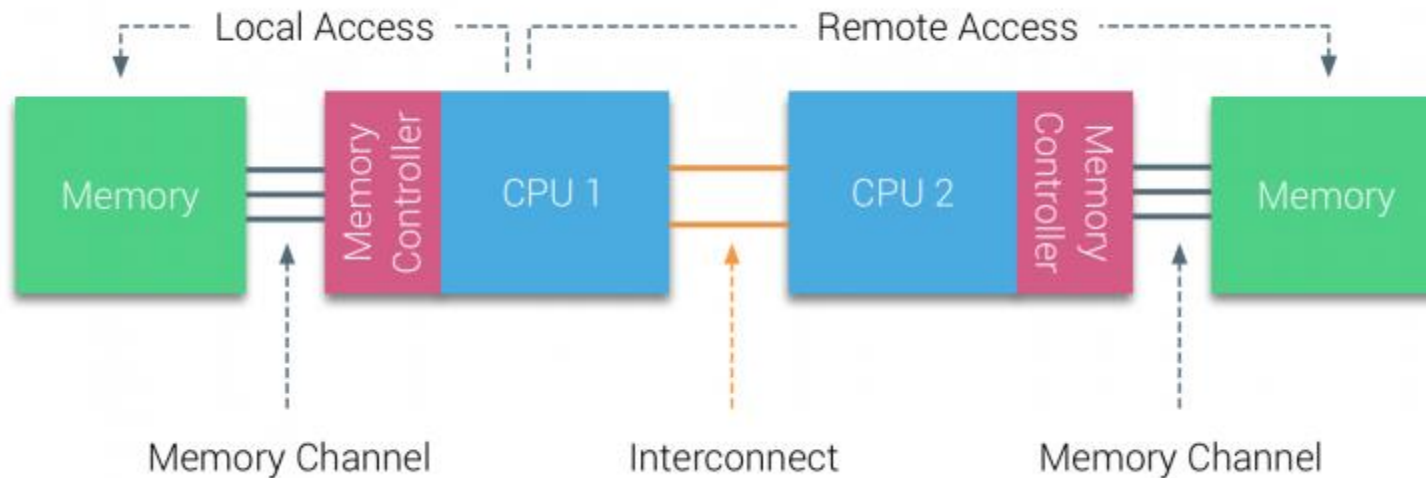
Use a higher bandwidth interconnection network



Uniform memory access architecture (UMA)

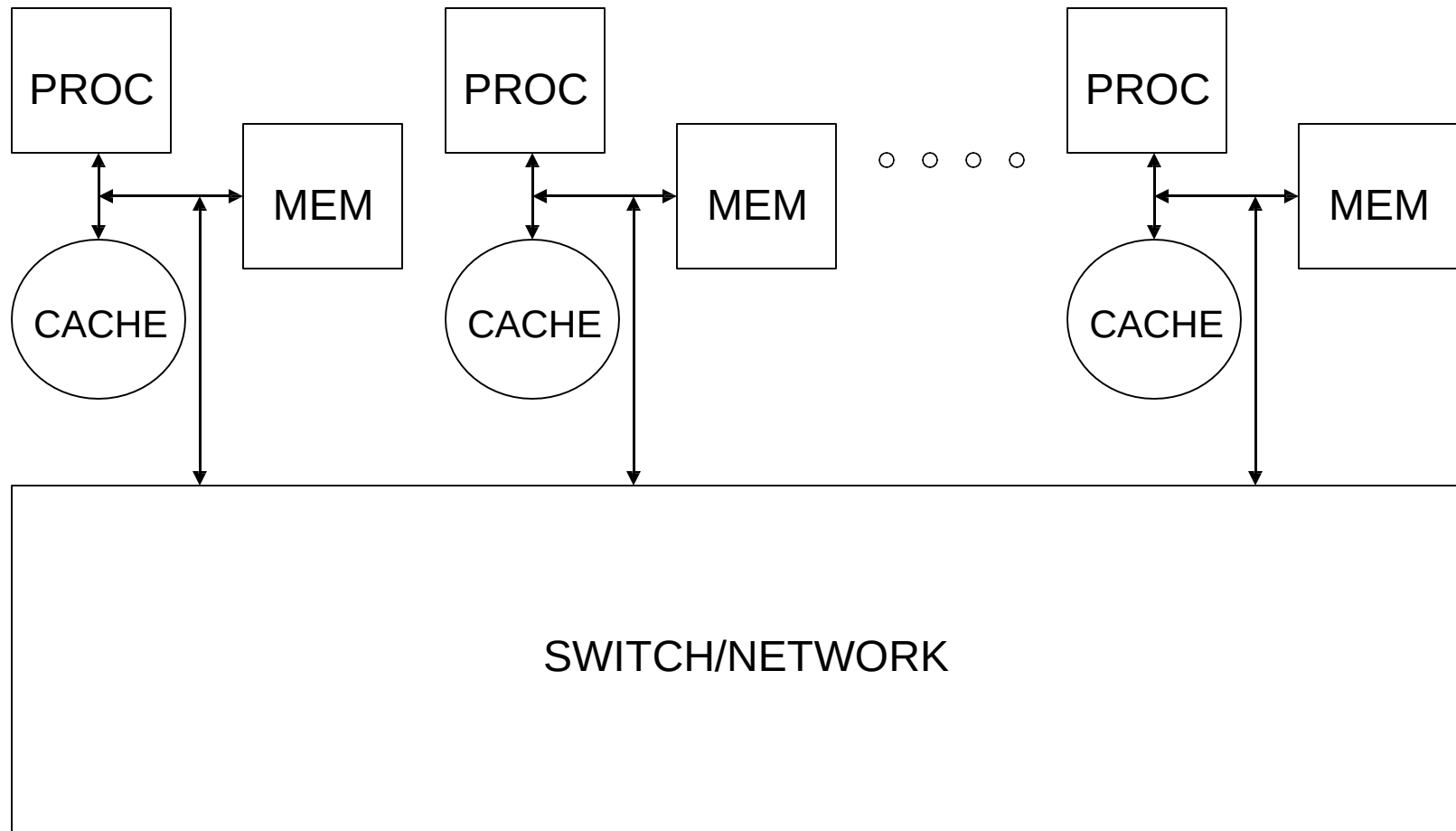
Distributed Shared-Memory (DSM) - Cont.

For lower latency: Non-Uniform Memory Access architecture (NUMA)



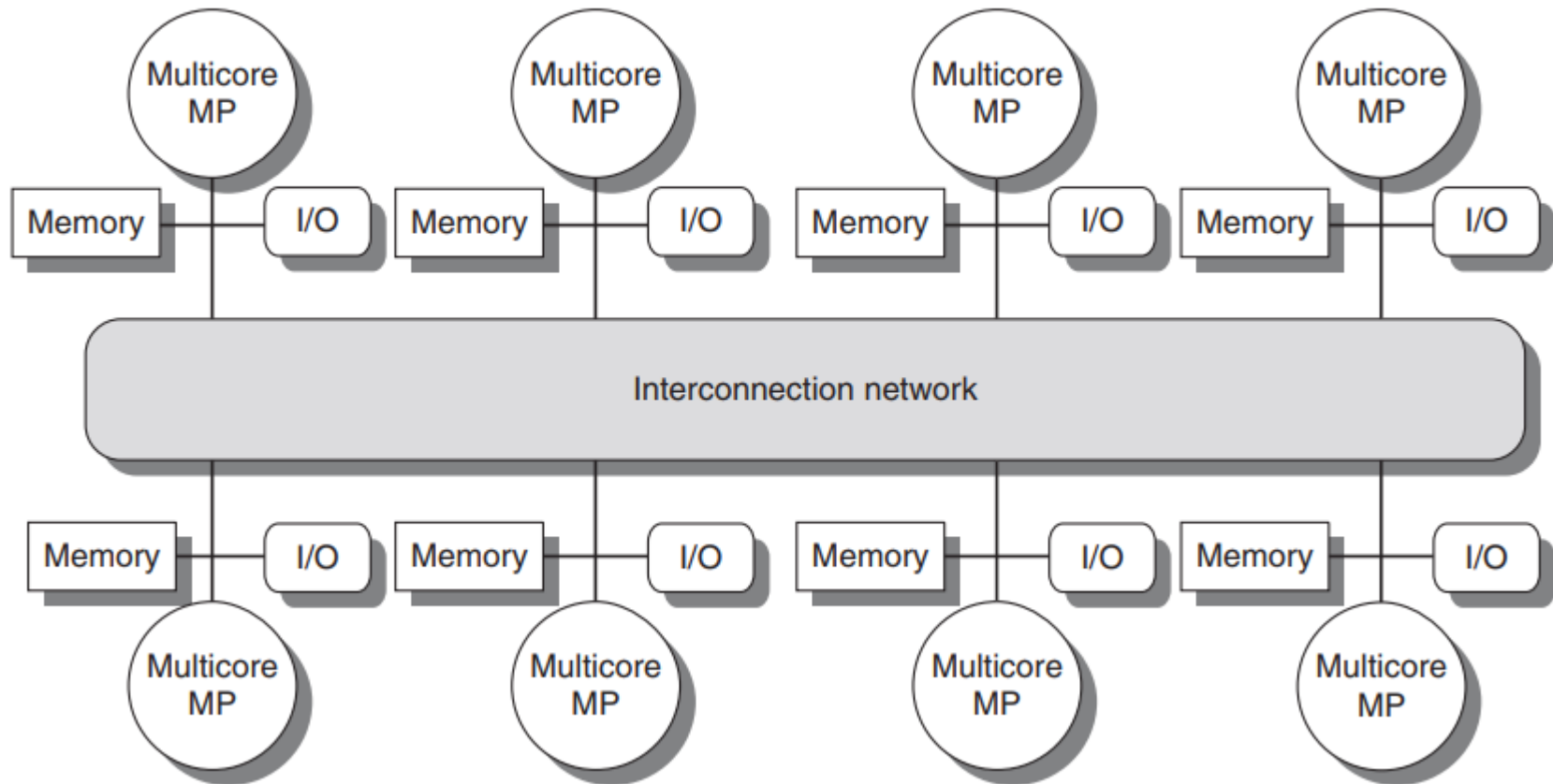
Non-Bus Interconnection Networks

Example interconnection networks



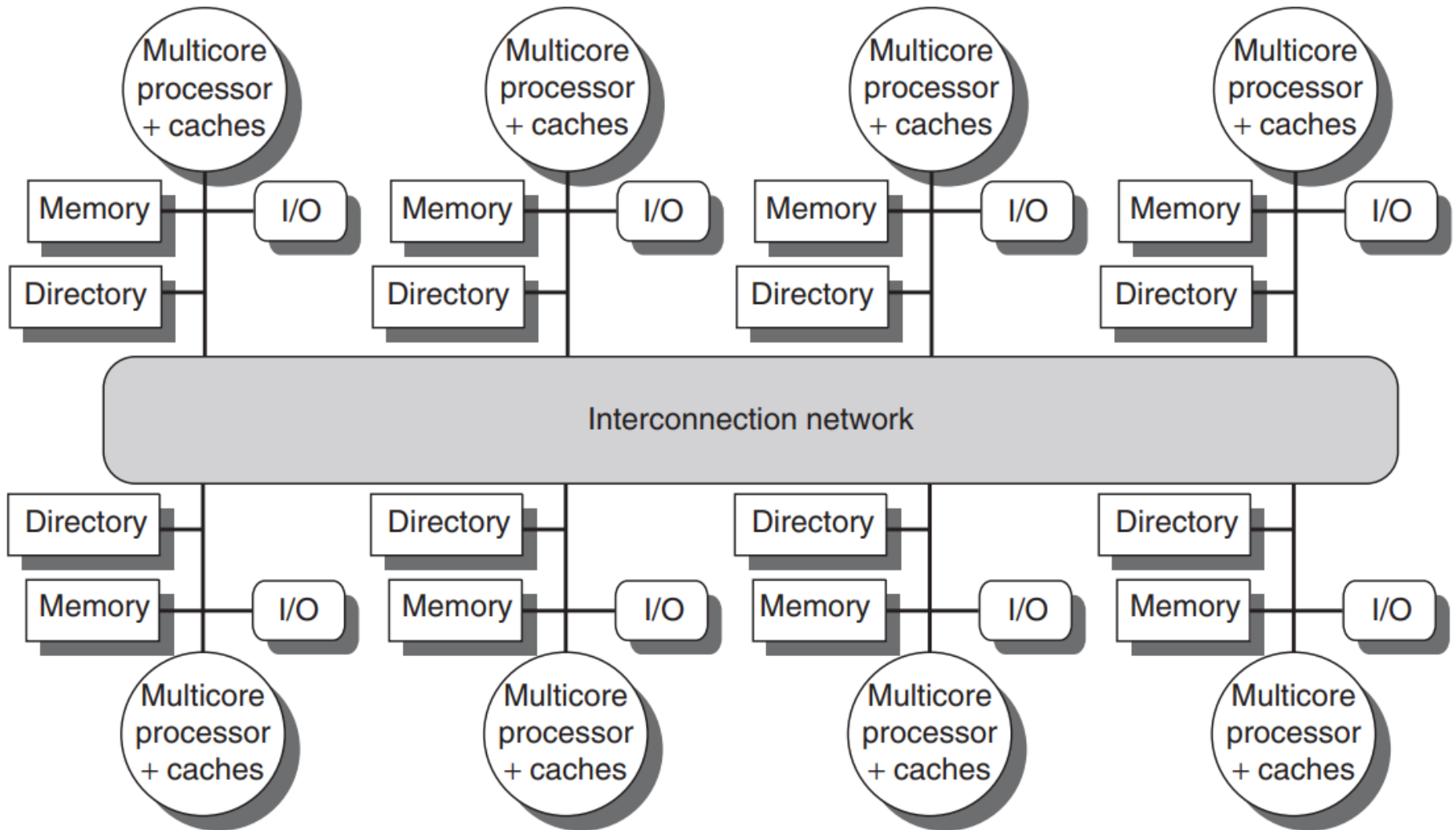
Distributed Shared-Memory - Coherence Problem

Directory scheme



Level of indirection!

Distributed Directory MPs



Directory Protocol

- Similar to Snoopy Protocol: Three states
 - Shared: ≥ 1 processors have data, memory up-to-date
 - Uncached (no processor has it; not valid in any cache)
 - Exclusive: 1 processor (**owner**) has data;
memory out-of-date
- In addition to cache state, must track which processors have data when in the shared state (usually bit vector, 1 if processor has copy)
- Keep it simple(r):
 - Writes to non-exclusive data
=> write miss
 - Processor blocks until access completes
 - Assume messages received
and acted upon in order sent

Directory Protocol

- No bus and don't want to broadcast:
 - interconnect no longer single arbitration point
 - all messages have explicit responses
- Terms: typically 3 processors involved
 - **Local node** where a request originates
 - **Home node** where the memory location of an address resides
 - **Remote node** has a copy of a cache block, whether exclusive or shared
- Example messages on next slide:
P = processor number, A = address

Directory Protocol Messages

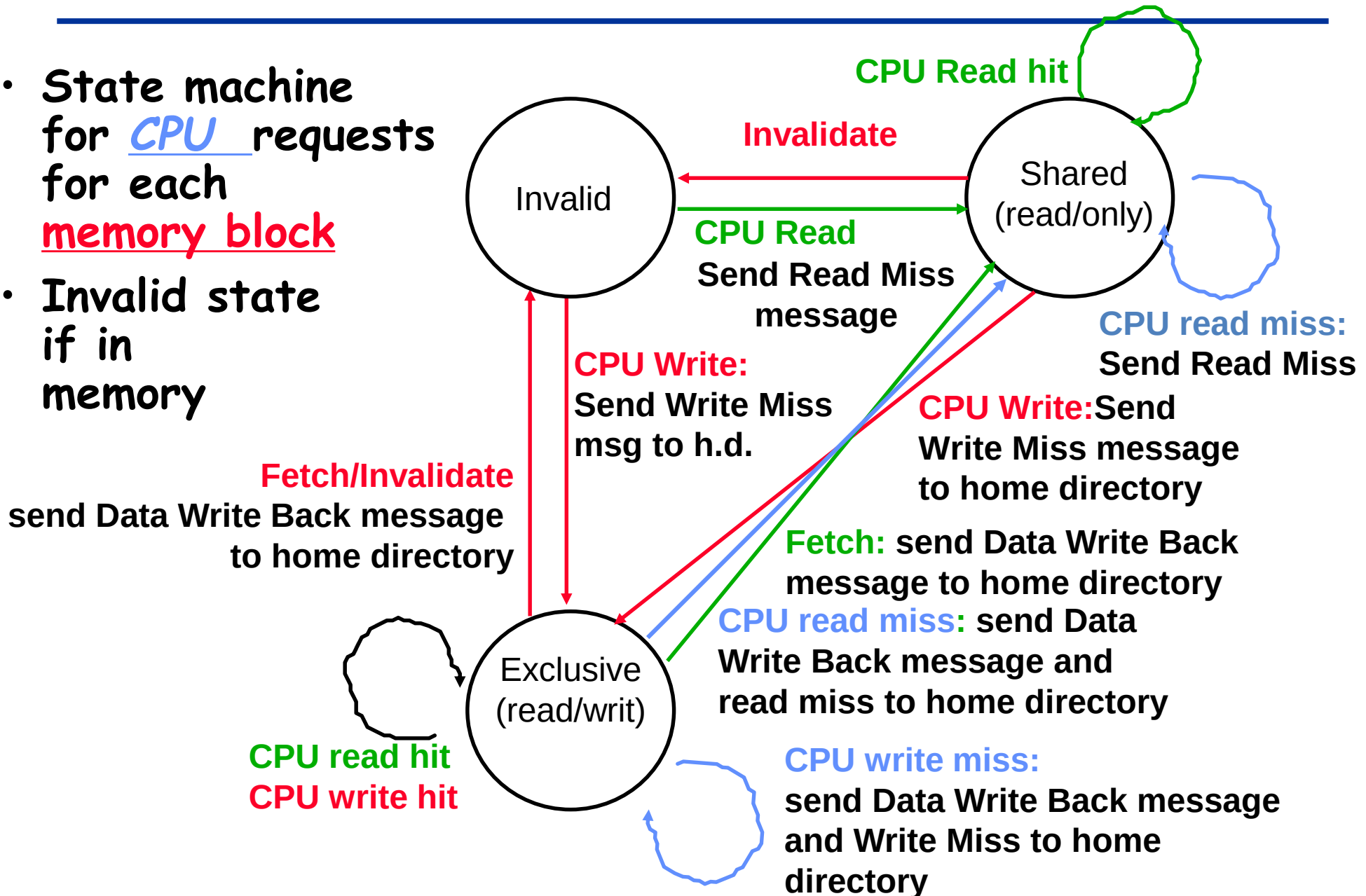
Message type	Source	Destination	Msg Content
Read miss	Local cache	Home directory	P, A
- Processor <i>P</i> reads data at address <i>A</i>; make <i>P</i> a read sharer and arrange to send data back			
Write miss	Local cache	Home directory	P, A
- Processor <i>P</i> writes data at address <i>A</i>; make <i>P</i> the exclusive owner and arrange to send data back			
Invalidate	Home directory	Remote caches	A
- Invalidate a shared copy at address <i>A</i>.			
Fetch	Home directory	Remote cache	A
- Fetch the block at address <i>A</i> and send it to its home directory			
Fetch/Invalidate	Home directory	Remote cache	A
- Fetch the block at address <i>A</i> and send it to its home directory; invalidate the block in the cache			
Data value reply	Home directory	Local cache	Data
- Return a data value from the home memory (read miss response)			
Data write-back	Remote cache	Home directory	A, Data
- Write-back a data value for address <i>A</i> (invalidate response)			

State Transition Diagram for an Individual Cache Block in a Directory Based System

- **States identical to snoopy case; transactions very similar.**
- **Transitions caused by read misses, write misses, invalidates, data fetch requests**
- **Generates read miss & write miss msg to home directory.**
- **Write misses that were broadcast on the bus for snooping => explicit invalidate & data fetch requests.**
- **Note: on a write, a cache block is bigger, so need to read the full cache block**

CPU -Cache State Machine

- State machine for CPU requests for each memory block
- Invalid state if in memory

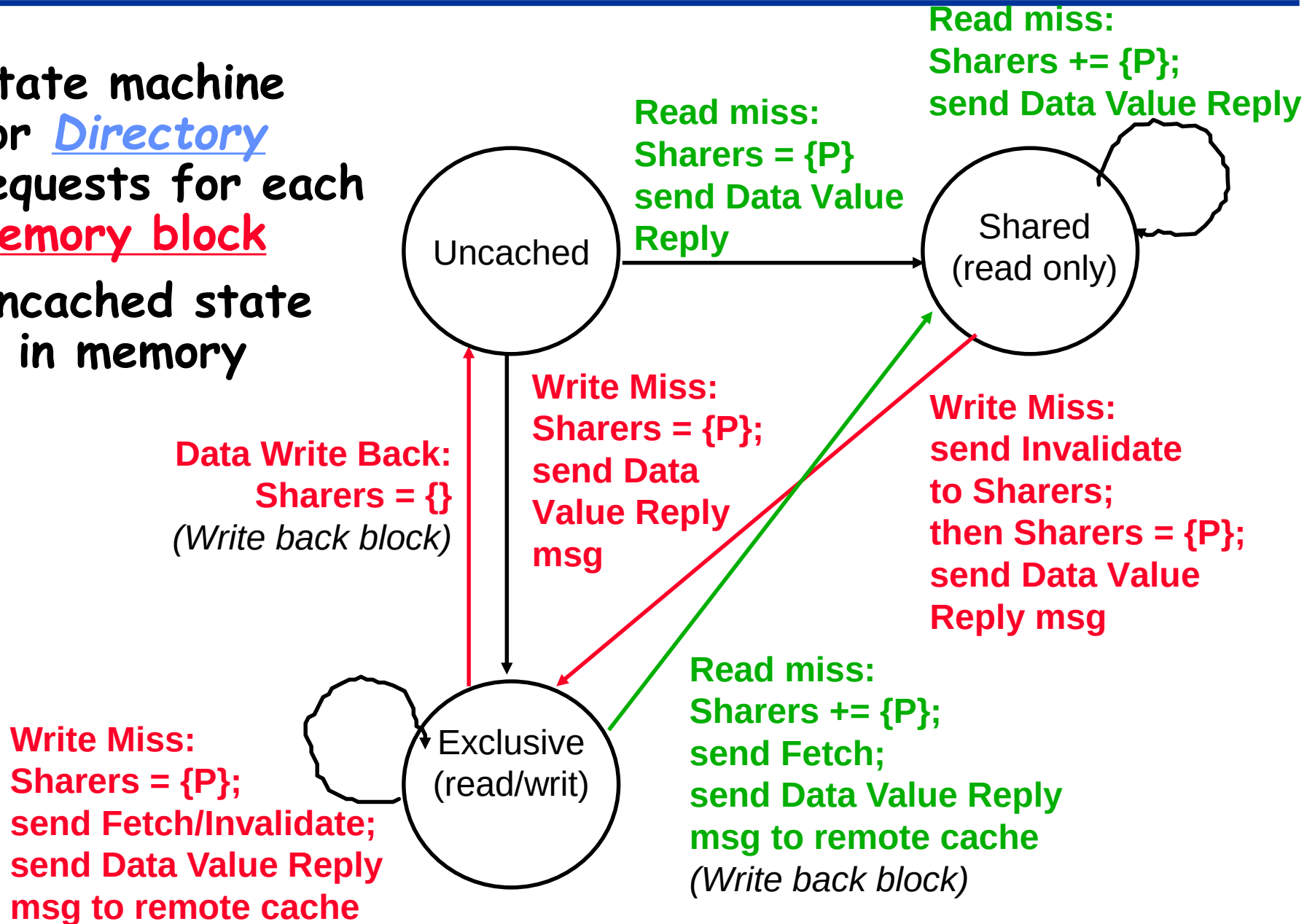


State Transition Diagram for the Directory

- **States identical to snoopy case; transactions very similar.**
- **Transitions caused by read misses, write misses, invalidates, data fetch requests**
- **Generates read miss & write miss msg to home directory.**
- **Write misses that were broadcast on the bus for snooping => explicit invalidate & data fetch requests.**
- **Note: on a write, a cache block is bigger, so need to read the full cache block**

Directory State Machine

- State machine for Directory requests for each memory block
- Uncached state if in memory



Example Directory Protocol

- **Message sent to directory causes two actions:**
 - Update the directory
 - More messages to satisfy request
- **Block is in **Uncached** state: the copy in memory is the current value; only possible requests for that block are:**
 - **Read miss:** requesting processor sent data from memory & requestor made only sharing node; state of block made Shared.
 - **Write miss:** requesting processor is sent the value & becomes the Sharing node. The block is made Exclusive to indicate that the only valid copy is cached. Sharers indicates the identity of the owner.
- **Block is **Shared** => the memory value is up-to-date:**
 - **Read miss:** requesting processor is sent back the data from memory & requesting processor is added to the sharing set.
 - **Write miss:** requesting processor is sent the value. All processors in the set Sharers are sent invalidate messages, & Sharers is set to identity of requesting processor. The state of the block is made Exclusive.

Example Directory Protocol

- Block is **Exclusive**: current value of the block is held in the cache of the processor identified by the set Sharers (the owner) => three possible directory requests:
 - **Read miss**: owner processor sent data fetch message, causing state of block in owner's cache to transition to Shared and causes owner to send data to directory, where it is written to memory & sent back to requesting processor. Identity of requesting processor is added to set Sharers, which still contains the identity of the processor that was the owner (since it still has a readable copy). State is shared.
 - **Data write-back**: owner processor is replacing the block and hence must write it back, making memory copy up-to-date (the home directory essentially becomes the owner), the block is now Uncached, and the Sharer set is empty.
 - **Write miss**: block has a new owner. A message is sent to old owner causing the cache to send the value of the block to the directory from which it is sent to the requesting processor, which becomes the new owner. Sharers is set to identity of new owner, and state of block is made Exclusive.

Example

Processor 1 Processor 2 Interconnect Directory Memory

	<i>P1</i>			<i>P2</i>			<i>Bus</i>			<i>Directory</i>			<i>Memory</i>	
<i>step</i>	<i>State</i>	<i>Addr</i>	<i>Value</i>	<i>State</i>	<i>Addr</i>	<i>Value</i>	<i>Action</i>	<i>Proc.</i>	<i>Addr</i>	<i>Value</i>	<i>Addr</i>	<i>State</i>	<i>{Procs}</i>	<i>Value</i>
P1: Write 10 to A1														
P1: Read A1														
P2: Read A1														
P2: Write 20 to A1														
P2: Write 40 to A2														

A1 and A2 map to the same cache block

Example: Step 1

Processor 1 Processor 2 Interconnect Directory Memory

	P1			P2			Bus			Directory			Memory	
step	State	Addr	Value	State	Addr	Value	Action	Proc.	Addr	Value	Addr	State	{Procs}	Value
P1: Write 10 to A1							WrMs	P1	A1		A1	Ex	{P1}	
	Excl.	A1	10				DaRp	P1	A1	0				
P1: Read A1														
P2: Read A1														
P2: Write 20 to A1														
P2: Write 40 to A2														

A1 and A2 map to the same cache block

Example: Step 2

Processor 1 Processor 2 Interconnect Directory Memory

	<i>P1</i>			<i>P2</i>			<i>Bus</i>			<i>Directory</i>				<i>Memory</i>
<i>step</i>	<i>State</i>	<i>Addr</i>	<i>Value</i>	<i>State</i>	<i>Addr</i>	<i>Value</i>	<i>Action</i>	<i>Proc.</i>	<i>Addr</i>	<i>Value</i>	<i>Addr</i>	<i>State</i>	<i>{Procs}</i>	<i>Value</i>
P1: Write 10 to A1							<i>WrMs</i>	P1	A1		<i>A1</i>	<i>Ex</i>	<i>{P1}</i>	
	<i>Excl.</i>	<i>A1</i>	<i>10</i>				<i>DaRp</i>	P1	A1	0				
P1: Read A1	Excl.	A1	10											
P2: Read A1														
P2: Write 20 to A1														
P2: Write 40 to A2														

A1 and A2 map to the same cache block

Example: Step 3

Processor 1 Processor 2 Interconnect Directory Memory

	P1			P2			Bus			Directory				Memory
step	State	Addr	Value	State	Addr	Value	Action	Proc.	Addr	Value	Addr	State	{Procs}	Value
P1: Write 10 to A1							WrMs	P1	A1		A1	Ex	{P1}	
	Excl.	A1	10				DaRp	P1	A1	0				
P1: Read A1	Excl.	A1	10											
P2: Read A1				Shar.	A1		RdMs	P2	A1					
	Shar.	A1	10				Ftch	P1	A1	10	A1			10
				Shar.	A1	10	DaRp	P2	A1	10	A1	Shar.	P1,P2}	10
P2: Write 20 to A1														
P2: Write 40 to A2														

Write Back

A1 and A2 map to the same cache block

Example: Step 4

Processor 1 Processor 2 Interconnect Directory Memory

	P1			P2			Bus				Directory			Memor
step	State	Addr	Value	State	Addr	Value	Action	Proc.	Addr	Value	Addr	State	{Procs}	Value
P1: Write 10 to A1							WrMs	P1	A1		A1	Ex	{P1}	
	Excl.	A1	10				DaRp	P1	A1	0				
P1: Read A1	Excl.	A1	10											
P2: Read A1				Shar.	A1		RdMs	P2	A1					
	Shar.	A1	10				Ftch	P1	A1	10	A1			10
				Shar.	A1	10	DaRp	P2	A1	10	A1	Shar.	P1,P2}	10
P2: Write 20 to A1				Excl.	A1	20	WrMs	P2	A1					10
	Inv.						Inval.	P1	A1		A1	Excl.	{P2}	10
P2: Write 40 to A2														

A1 and A2 map to the same cache block

Example: Step 5

Processor 1 Processor 2 Interconnect Directory Memory

	P1			P2			Bus				Directory			Memory
step	State	Addr	Value	State	Addr	Value	Action	Proc.	Addr	Value	Addr	State	{Procs}	Value
P1: Write 10 to A1							WrMs	P1	A1		A1	Ex	{P1}	
	Excl.	A1	10				DaRp	P1	A1	0				
P1: Read A1	Excl.	A1	10											
P2: Read A1				Shar.	A1		RdMs	P2	A1					
	Shar.	A1	10				Frch	P1	A1	10	A1			10
				Shar.	A1	10	DaRp	P2	A1	10	A1	Shar.	P1,P2}	10
P2: Write 20 to A1				Excl.	A1	20	WrMs	P2	A1					10
	Inv.						Inval.	P1	A1		A1	Excl.	{P2}	10
P2: Write 40 to A2							WrMs	P2	A2		A2	Excl.	{P2}	0
							WrBk	P2	A1	20	A1	Unca.	{}	20
				Excl.	A2	40	DaRp	P2	A2	0	A2	Excl.	{P2}	0

A1 and A2 map to the same cache block

Acknowledgements

These slides contain material developed and copyright by:

- Sarita Adve (UIUC)
- Nathan Beckmann (CMU)
- David Patterson (UC Berkeley)
- Yuanchun Zhang (NUDT)
- Qianni Deng (SJTU)